

Advanced Database Design Note

Name :

Roll No:

Semester:

Contact :

College :

Index

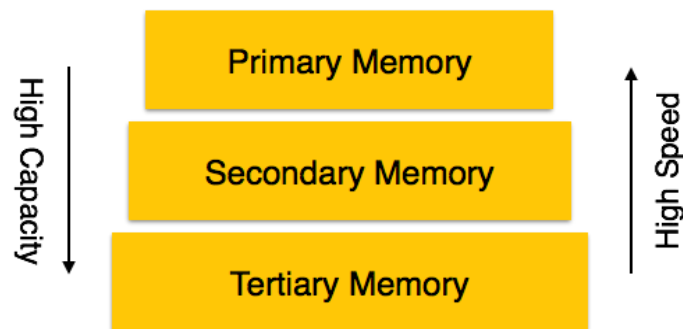
Unit	Title	Page No.
1	Storage and File Structures	2
2	Indexing and Hashing	16
3	Query Processing	24
4	Query Optimization	33
5	Transaction Management	39
6	Concurrency Control	44
7	Recovery System	51
–	Missing QNs Topic	57

Unit 1

Storage and File Structures

Physical Storage Media:

Databases are stored as files that contain records. At the physical level, data is stored in electronic or electromagnetic form on different storage devices. These devices are generally classified into three types: Primary storage, Secondary storage, and Tertiary storage.



Primary Storage:

Primary storage is the memory that is directly accessible by the CPU and is used for immediate processing of data. It includes CPU registers, cache memory, and main memory (RAM), which are located on the motherboard or inside the CPU chip. This type of storage is very fast, allowing quick access to instructions and data currently being used by the system.

However, it has a small capacity compared to other storage types. Primary storage is also volatile, meaning all data is lost when the power is switched off. Because of its speed, it is mainly used to hold active programs and data that are currently being processed.

Secondary Storage:

Secondary storage is used for long-term or permanent storage of data. Unlike primary storage, it is not directly accessible by the CPU and requires input/output operations to access data. It includes devices such as hard disk drives (HDD), solid-state drives (SSD), USB flash drives, optical discs (CD/DVD), and magnetic tapes.

Secondary storage has a much larger capacity compared to primary storage but is relatively slower in speed. It is non-volatile, meaning data remains stored even when the system is powered off. In DBMS, secondary storage is the main location where database files are stored permanently.

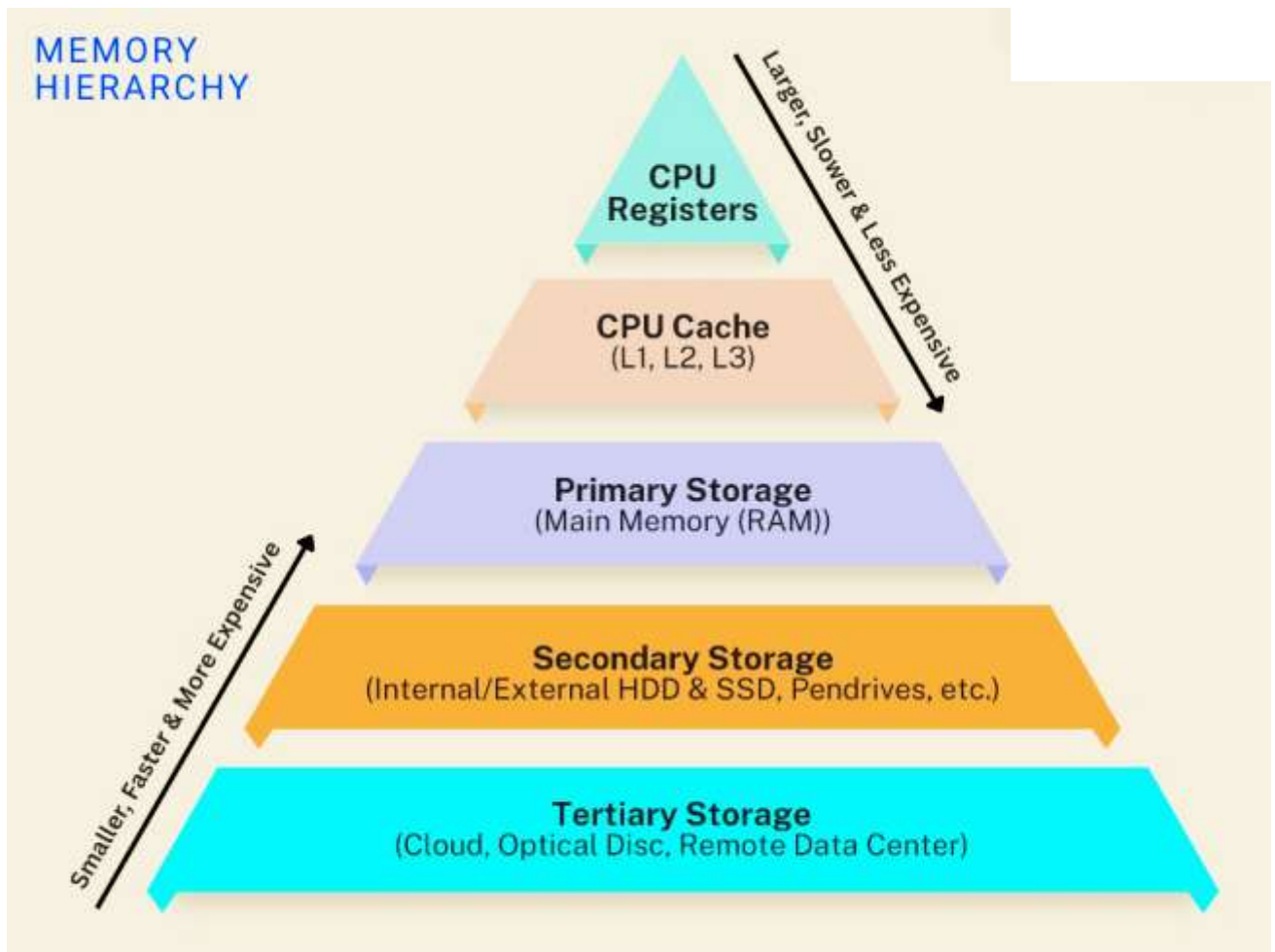
Tertiary Storage:

Tertiary storage is used for storing very large amounts of data for a long time, mainly as backup. It is outside the main computer system and is the slowest type of storage. It usually uses devices like magnetic tapes or optical discs, often managed automatically in large storage systems. This storage is not used for daily work; instead, it is used when data is rarely needed. Organizations mainly use it for backup, disaster recovery, and long-term storage.

Memory Hierarchy:

Memory hierarchy is a way of organizing computer memory into different levels based on speed, cost, and storage size. The fastest memory is very small and expensive, while slower memory is larger and

cheaper. This layered structure helps the CPU access data efficiently by keeping frequently used data in faster memory and less-used data in slower storage. The memory hierarchy diagram is given as:



Registers:

Registers are the smallest and fastest memory units inside the CPU. They are used to store data and instructions that are currently being processed. Registers can hold values like instructions, memory addresses, or small data items. Because they are inside the CPU, access is extremely fast. However, registers have very limited storage capacity and are used only for immediate processing tasks.

Cache Memory:

Cache memory is a very fast memory placed between the CPU and RAM. It stores frequently used data and instructions so the CPU can access them quickly without going to main memory every time. Cache is faster than RAM but smaller in size and more expensive. Its main purpose is to reduce data access time and improve CPU performance by keeping commonly used information close to the processor.

Main Memory (RAM):

Main memory, also called RAM (Random Access Memory), is where programs and data are stored when they are actively used by the CPU. When you open a program, it is loaded from secondary storage into RAM for faster processing. RAM is faster than storage devices but slower than cache. It is also volatile, meaning data is lost when power is turned off. ROM (Read Only Memory) is another type of primary memory, but it is non-volatile and stores permanent instructions.

Secondary Storage (Disk Storage):

Secondary storage is used for permanent data storage. It includes devices like hard disk drives (HDD), solid-state drives (SSD), CDs, DVDs, and flash drives. Unlike RAM, data stored here is not lost when the power is off. It is much larger and cheaper than main memory but slower in speed. Data must be transferred from secondary storage to RAM before the CPU can use it.

Tertiary Storage (Tape Storage):

Tertiary storage is used for very large-scale backup and long-term storage. It includes devices like magnetic tapes and tape libraries. It is the slowest type of storage, but it is very cheap and can store huge amounts of data. It works on sequential access, meaning data is read in order, not randomly. Because of this, it is mainly used for backup, archiving, and disaster recovery rather than daily use.

Physical Characteristics of Disk:

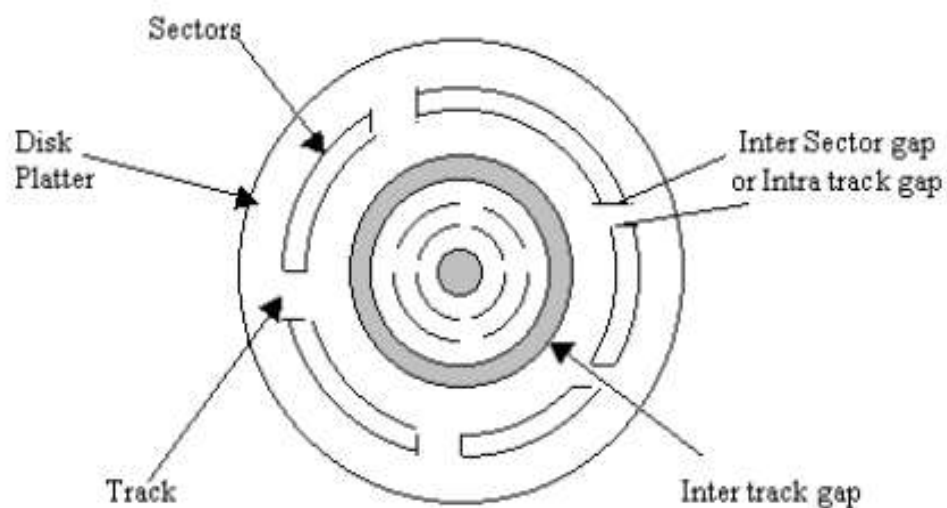


Fig: Disk Plate Block diagram

- ✚ A magnetic disk is a circular storage device made of plastic or metal, coated with a magnetic material (like iron oxide) on both sides.
- ✚ Data is stored on the disk surface in the form of tiny magnetic spots, where magnetized spots represent 1s and non-magnetized spots represent 0s.
- ✚ A read/write head is used to read and write data on the disk surface.
- ✚ In most cases, the head remains stationary while the disk rotates underneath it during read and write operations.
- ✚ The disk surface is divided into concentric circles called tracks.
- ✚ Each disk has a different number of tracks depending on its design.
- ✚ Each track is further divided into smaller units called sectors.
- ✚ Every sector has a fixed storage capacity.
- ✚ Each sector is assigned a unique number for identification.

Performance Measures of Disk:

The performance and quality of a disk are mainly measured by its access time, data transfer rate, and reliability.

1. Access Time:

Access time is the total time taken by a disk to locate the required data and start transferring it after a read or write request is made.

To access data, the read/write head first moves to the correct track. The time taken for this movement is called seek time, which usually ranges from 2 to 30 milliseconds. After reaching the correct track, the disk rotates until the required sector comes under the head. This waiting time is called rotational latency time.

Therefore, Total access time = Seek Time + Rotational Latency Time

(Typical access times range from 10 to 40 milliseconds.)

2. Data Transfer Rate:

Data transfer rate is the speed at which data can be read from or written to the disk after access is obtained. It is usually measured in megabytes per second (MB/s). A higher transfer rate means data can be moved more quickly between the disk and main memory, improving overall system performance.

3. Reliability:

Reliability shows how dependable a disk is during operation. It is measured using Mean Time to Failure (MTTF), which is the average estimated time a disk can work before failure occurs. Modern disks usually have an MTTF ranging from 30,000 to 800,000 hours (about 3.4 to 91 years). A higher MTTF indicates a more reliable disk with a lower chance of failure or data loss.

Optimization of Disk Block Access:

Data is transferred between disk and main memory in units called blocks. A block is a contiguous set of bytes stored on a single disk track, and its size usually ranges from 512 bytes to a few KB. Since disk access is much slower than main memory, following techniques are used to optimize disk access.

1. Disk Scheduling

If multiple disk requests are pending, the disk controller does not process them in random order. Instead, it arranges them so that the disk head moves in a smooth and shortest path, serving nearby requests together. This reduces unnecessary movement, reduces seek time, and improves performance.

2. File Organization

File organization means how data is arranged on the disk. If related data is stored close together, the disk head does not need to move much to access it. This reduces access time and improves performance.

3. Non-volatile write buffer:

If a write request is made, then data is first stored in a fast non-volatile memory (like battery-backed RAM). The system immediately confirms the write, and the actual writing to disk is done later. This increases write speed and improves performance.

4. Log Disk

If a log disk is used, then all write operations are stored sequentially one after another on a separate disk. Since data is written in order without random access, it removes most seek time and makes writing much faster.

RAID (Redundant Array of Independent Disks):

[V-IMP FOR EXAM]

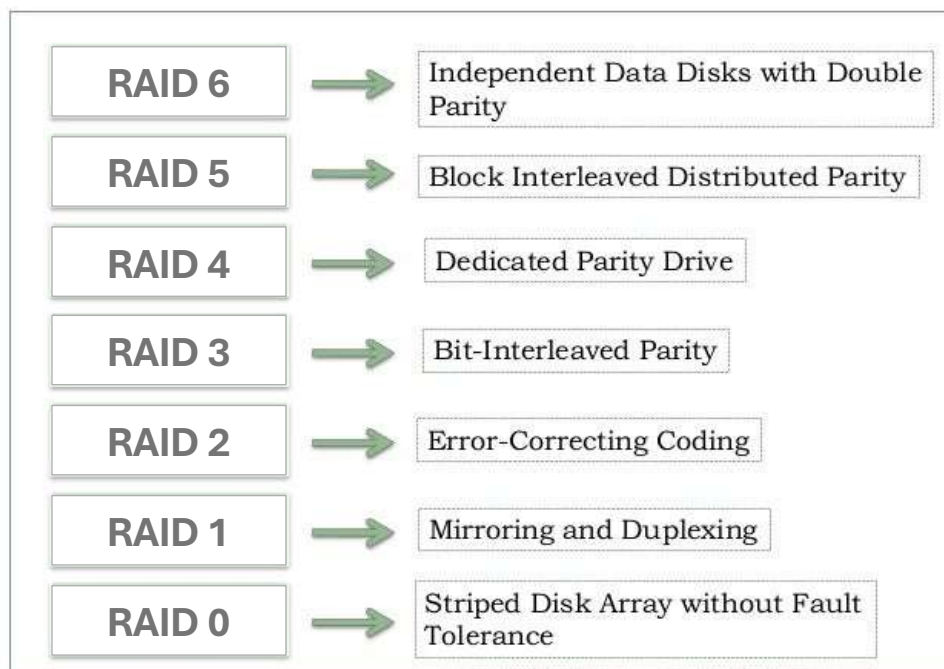
RAID is a method of combining multiple physical disks into one logical system to improve speed, reliability, or both. It works by storing data across disks using techniques like striping (splitting data), mirroring (copying data), and parity (error-checking information).

The benefits / advantages of RAID are:

- RAID improves overall disk performance by allowing multiple disks to work together, increasing read and write speed.
- It provides data reliability and fault tolerance, so data is not easily lost if a disk fails (depending on RAID level).
- It increases data availability, meaning the system can continue working even if one disk fails (in some RAID levels like RAID 1, 5, 6).
- RAID allows better storage efficiency and management compared to single large disks.
- It supports faster data access by distributing data across multiple disks (parallel access).
- It helps in data protection and backup, reducing the risk of permanent data loss.

The different levels of RAID are:

[First 3 level – 0, 1, 2 Very IMP for Exam]



RAID 0 (Striping):

RAID 0 splits data into blocks and stores them across multiple disks to improve speed. It focuses only on performance, not safety.

Example:

File blocks A, B, C, D are stored as:

- ❖ Disk 1 → A, C
- ❖ Disk 2 → B, D

Pros:

- ✚ Very fast read and write speed because multiple disks work at the same time
- ✚ High performance for large data transfer

Cons:

- ✚ No data protection
- ✚ If one disk fails, all data is lost

RAID 1 (Mirroring):

RAID 1 stores an exact copy of data on two or more disks. It focuses on data safety.

Example:

File blocks A, B, C, D are stored as:

- ❖ Disk 1 → A, B, C
- ❖ Disk 2 → A, B, C (exact copy)

Pros:

- ✚ Very high reliability and data safety
- ✚ Data is still available even if one disk fails
- ✚ Fast read performance (data can be read from either disk)

Cons:

- ✚ 50% storage capacity is effectively wasted (because everything is duplicated)
- ✚ More expensive because extra disks are required

RAID 2 (Bit-Level Striping with ECC):

RAID 2 splits data at bit level and stores in multiple disks. Along with data disks, it uses extra disks to store Error Correcting Codes (ECC), which are used to detect and correct errors in data.

Example:

Suppose we want to store a binary data word: 1011. Then,

- ❖ Disk 1 → 1
- ❖ Disk 2 → 0
- ❖ Disk 3 → 1
- ❖ Disk 4 → 1
- ❖ Disk 5 → stores ECC (error-correcting bits for checking and recovery)

If any bit gets corrupted in one disk, the ECC disk helps to detect and correct the error.

Pros:

- ✚ Very strong error detection and correction using ECC
- ✚ High reliability because errors can be recovered

Cons:

- ✚ Very complex design and requires special hardware support
- ✚ Slow and inefficient for normal use due to bit-level operations and multiple ECC disks
- ✚ Rarely used in modern systems because better RAID levels exist (like RAID 5/6)

RAID 3 (Byte-Level Striping with Parity Disk):

RAID 3 divides data into bytes and stores them across multiple disks. One separate disk is used to store parity information, which helps recover data if one disk fails. It is mainly designed for fast sequential data transfer.

Example:

Suppose a file is divided into bytes: **A, B, C, D**

- ❖ Disk 1 → A
- ❖ Disk 2 → B
- ❖ Disk 3 → C
- ❖ Disk 4 → D
- ❖ Disk 5 → Parity (used to recover data if any one disk fails)

If one disk fails, the missing data can be rebuilt using parity information.

Pros:

- 🚦 Good performance for large sequential data transfer
- 🚦 Can recover data if one disk fails (fault tolerance)
- 🚦 Efficient for reading large files

Cons:

- 🚦 Parity disk becomes a bottleneck (slows down writes)
- 🚦 Poor performance for multiple small or random requests
- 🚦 Not commonly used in modern systems

Storage Access:

A database is stored as a collection of files managed by the operating system. These files are further divided into blocks, and each block can store one or more data items. The main goal of a DBMS is to reduce disk access (block transfers between disk and main memory) because disk operations are very slow. Since all data cannot fit in main memory at once, the DBMS decides which blocks should be kept in memory and which should remain on disk.

This process is similar to memory management done by the operating system. However, sometimes the DBMS and OS may conflict because both try to control how memory and disk space are used for data storage.

Buffer Manager:

The buffer manager is a part of the DBMS that manages main memory space (buffer) used for storing database blocks temporarily. It works as:

- 🚦 It handles all requests for reading or writing database blocks.
- 🚦 If the required block is already in main memory, it directly gives its memory address.
- 🚦 If the block is not in memory, it is brought from disk into the buffer. If the buffer is full, another block is removed to make space.
- 🚦 This helps reduce disk access and improves performance.

Buffer Replacement Strategy:

[or, Buffer Replacement Policies]

When the buffer is full and a new block needs to be loaded, the DBMS must decide which existing block to remove. This decision method is called a buffer replacement strategy. The buffer replacement strategies are:

1. Least Recently Used (LRU):

In the LRU strategy, when the buffer is full, the block that has not been used for the longest time is replaced first. The idea behind this is that data which was used recently is more likely to be used again, so older unused data can be removed safely. This is one of the most commonly used strategies in DBMS for managing buffer space efficiently.

2. Toss-Immediate Strategy:

In the Toss-Immediate strategy, a block is removed from the buffer immediately after it is no longer needed. Once the DBMS finishes using a block during processing, it is freed right away instead of keeping it in memory. This helps to quickly release buffer space so that new blocks can be loaded without waiting for a replacement decision.

3. Most Recently Used (MRU):

In the MRU strategy, the block that was used most recently is replaced first. This works opposite to LRU. It is used in situations where recently accessed data is unlikely to be needed again soon. Although it is not commonly used in general database systems, it can be useful in specific access patterns.

4. First In First Out (FIFO):

The block that entered the buffer first is removed first, regardless of usage. It is simple but not very efficient because it does not consider how often a block is used.

5. Least Frequently Used (LFU):

The block that is used the least number of times is replaced first. It assumes that rarely used data is less important.

File Organization:

File organization refers to how records are arranged and stored in a file on disk blocks, and how they are accessed in a DBMS. It defines the logical structure of data storage so that records can be efficiently stored and retrieved using a primary key. It is needed because it improves data access speed, reduces search time, and helps manage large amounts of data stored on disk efficiently. Without proper file organization, data retrieval would be slow and inefficient.

The main types of file organization are:

-  Sequential file organization
 - Pile File Method (Unsorted Sequential File)
 - Sorted File Method (Sorted Sequential File)
-  Heap file organization
-  Hash file organization
-  B+ Tree file organization.

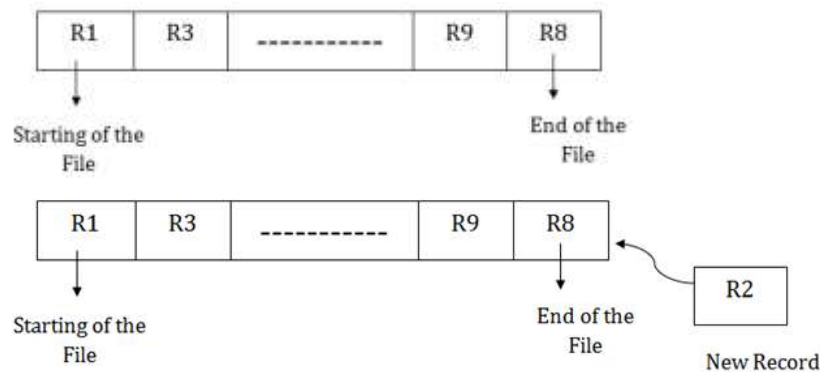
Sequential File Organization:

Sequential file organization is the simplest method of storing records in a file. In this method, records are stored one after another in a sequence on disk, usually in the order they are inserted into the database.

Pile File Method (Unsorted Sequential File):

In the pile file method, records are stored in the order they are received, without any sorting. Each new record is simply added at the end of the file. For example, if records are stored as R1, R3, R9, R8, and a new record R2 is inserted, it will be placed at the end of the file.

When a record needs to be updated or deleted, the system must search through the file sequentially until it finds the required record. After finding it, the record is either marked for deletion or updated and rewritten.

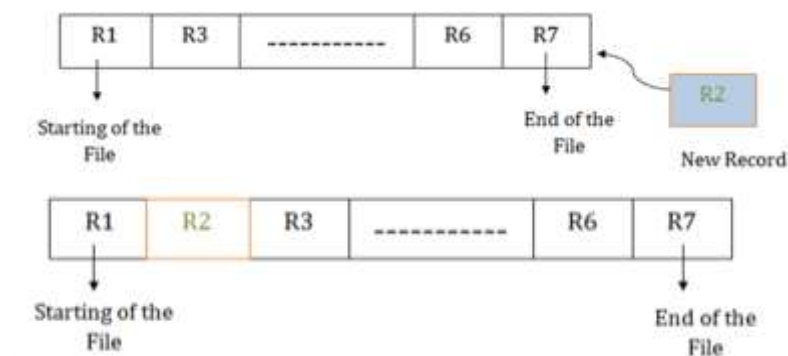


Sorted File Method (Sorted Sequential File):

In the sorted file method, records are stored in a file in a sorted order (ascending or descending) based on a key field such as the primary key. When a new record is inserted, it is first added at the end of the file and then the file is rearranged to maintain the correct sorted order. If any record is modified, it is updated and then the file is again sorted so that the order is maintained correctly.

Example:

Suppose records are already stored in sorted order as: R1, R3, R4, R5, R6, R7. If a new record R2 is inserted, it is first added at the end of the file and then the entire file is rearranged to maintain order: R1, R2, R3, R6, R7.



Pros of Sequential File Organization:

- Simple and easy to design and implement
- Efficient for processing large amounts of data in sequence
- Suitable for storage on low-cost devices like magnetic tapes
- Useful when most records need to be accessed together (e.g., reports, salary processing, grade calculation)

Cons of Sequential File Organization:

- Slow access for individual records because searching is sequential
- No direct access; each record must be checked one by one
- In sorted files, insertion and updates are slow due to re-sorting after changes

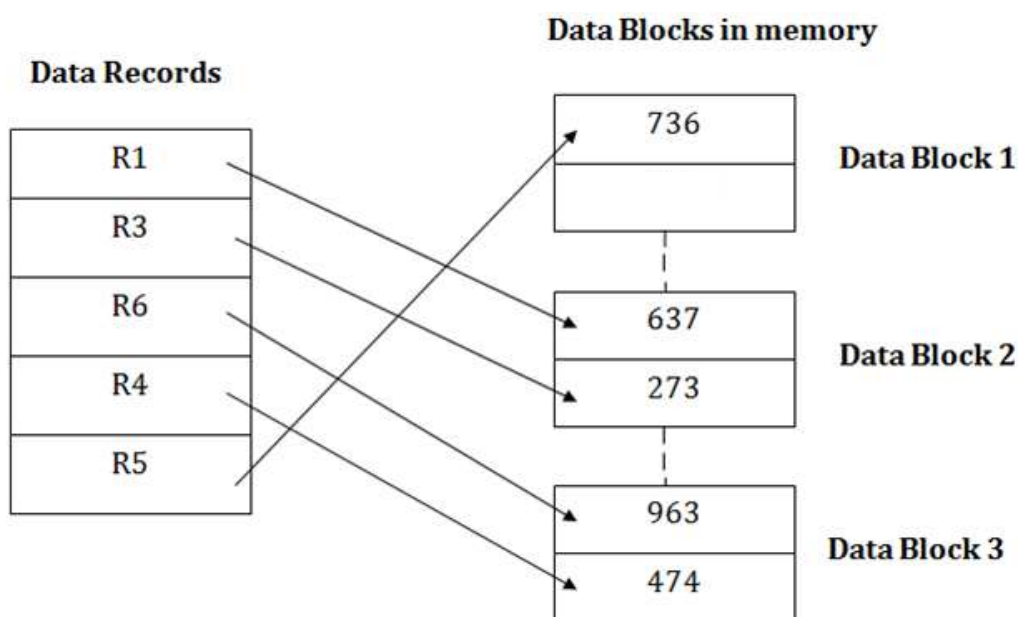
Heap File Organization:

Heap file organization is the simplest method of storing records in a DBMS. In this method, records are stored in no particular order and are inserted directly into any available space in disk blocks. When a block becomes full, new records are placed in another free block. The DBMS manages where each record is stored, and each record is identified using a unique ID.

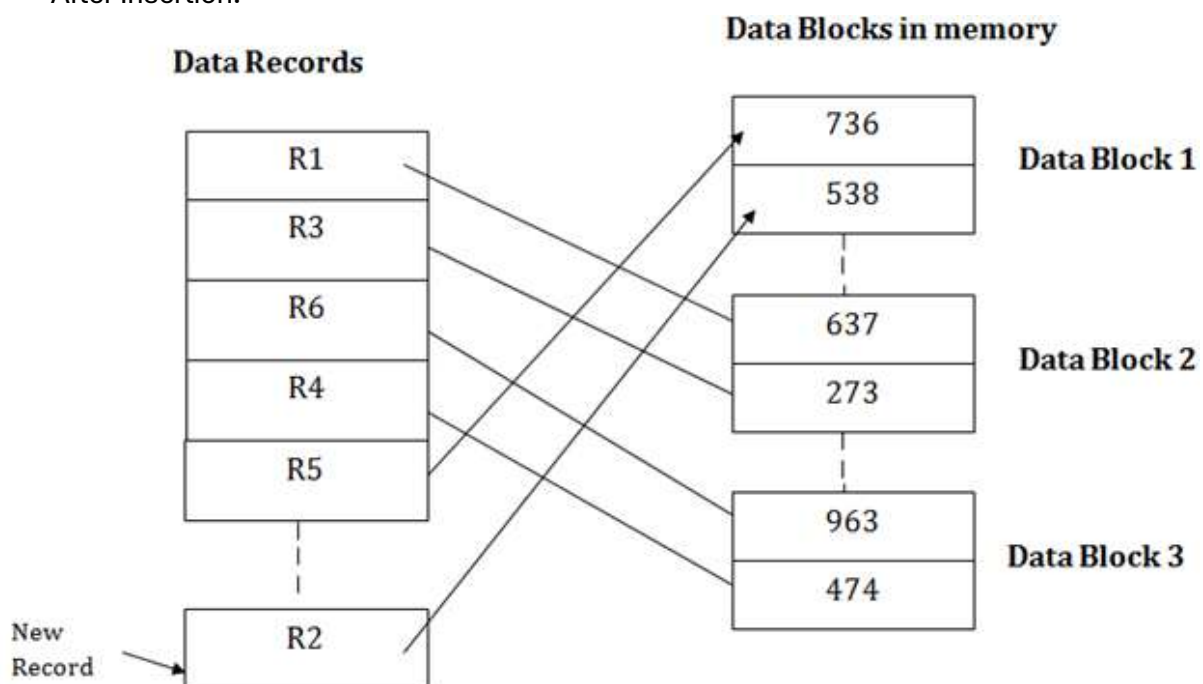
To find a record in a heap file, the system must search from the beginning and check each record one by one until the required record is found. Since there is no ordering, operations like searching, updating, or deleting become slow in large databases.

Example:

Suppose records R1, R3, R6, R4, and R5 are stored in a heap file. If a new record R2 is inserted, and block 3 is full, then the DBMS may place R2 in any other available block, such as block 1, without following any order.



After Insertion:



Pros of Heap File Organization:

- 🚧 Very good for **bulk insertion** because records can be inserted quickly without sorting.
- 🚧 Simple structure, so it is easy to load large amounts of data into the database.
- 🚧 Works efficiently for **small databases**, where searching is not very costly.

Cons of Heap File Organization:

- 🚧 Very slow for **searching, updating, and deleting** records because all records must be scanned.
- 🚧 Inefficient for **large databases**, since there is no ordering or indexing.
- 🚧 Performance decreases as data size increases.

Hash File Organization:

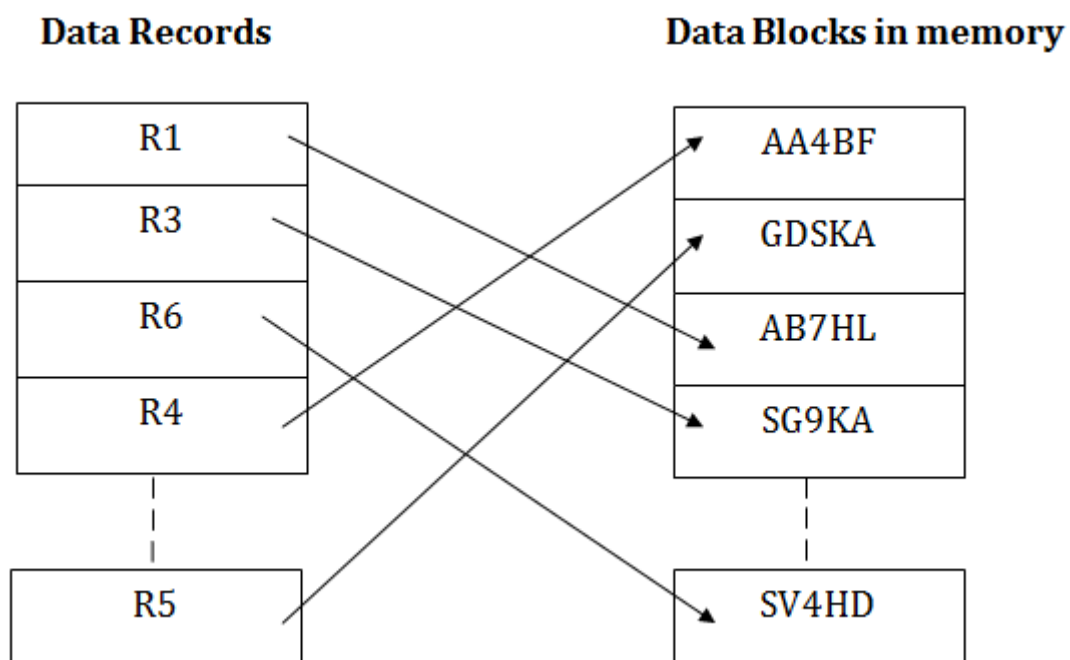
Hash file organization is a method in which a hash function is applied to a key field (called hash key) to determine the exact location (disk block) where a record will be stored. The output of the hash function directly gives the address of the storage block, so records are placed and accessed quickly.

When a record needs to be inserted, the hash function is applied to its key, and the record is stored directly in the computed block. Similarly, for searching, updating, or deleting a record, the same hash function is used to directly locate the record without scanning the entire file. This makes access very fast compared to sequential or heap file organization.

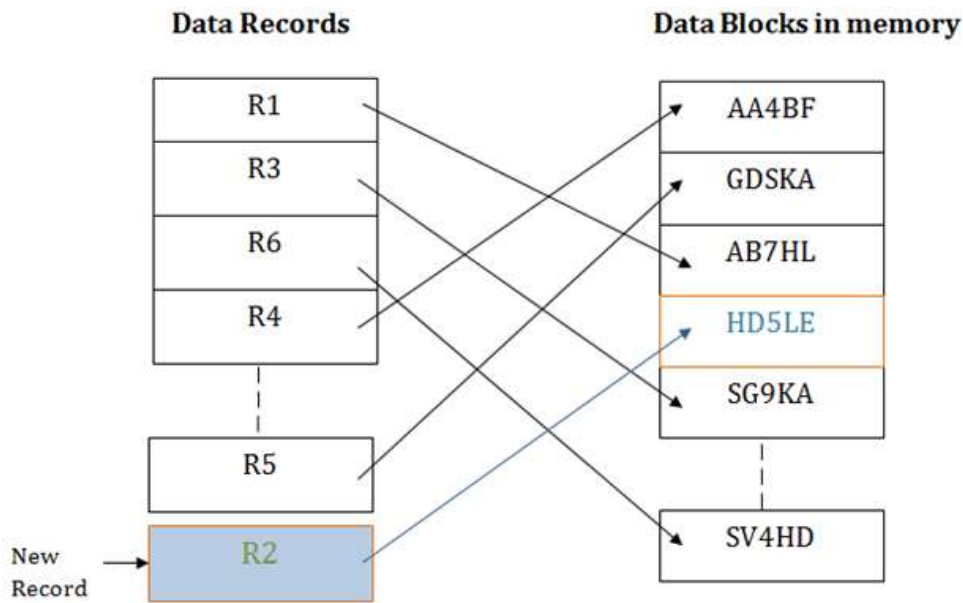
However, since multiple records may produce the same hash value, collisions can occur, which must be handled using techniques like chaining or overflow blocks. Records are stored in a seemingly random order based on the hash function.

Example:

Before inserting R2



To add R2, we apply hash function on R2 Key, and get the address as a result. So, we store R2 block in this address which is shown below.



Pros of Hash File Organization:

- ✚ Very fast **direct access** to records using hash function
- ✚ No need to search through all records
- ✚ Efficient for **exact match queries** (like searching by ID)
- ✚ Fast insertion, update, and deletion operations

Cons of Hash File Organization:

- ✚ Collisions can occur (different records may map to the same location)
- ✚ Performance decreases if collisions are high
- ✚ Not suitable for **range queries** (Ex: finding records between 10–50)
- ✚ Difficult to manage when data grows dynamically

B+ File Organization:

B+ tree file organization is an advanced indexing method used in DBMS to store and access records efficiently. It organizes data in a balanced tree structure based on a key (usually the primary key), which helps in fast searching and retrieval.

In this structure, the tree has a root node, intermediate (index) nodes, and leaf nodes. The important point is that only leaf nodes store actual data records, while internal nodes only store keys to guide the search path. All leaf nodes are also linked in sorted order, which makes both direct and sequential access efficient.

To search a record, the system starts from the root and moves step by step through intermediate nodes until it reaches the correct leaf node. Because the tree is balanced, the search takes very few steps even for large datasets.

Example:

In the given B+ tree, the root node contains 25, which is used to start the search. The intermediate nodes do not store actual data; they only contain pointers to guide the search. Values smaller than 25 (like 15) are on the left side, and values greater than 25 (like 30) are on the right side of the root node.

The actual records are stored only in the leaf nodes in sorted order, such as 10, 12, 17, 20, 24, 27, and 29. Since all leaf nodes are balanced and linked, any record can be found quickly by following a single path from the root to the correct leaf node.

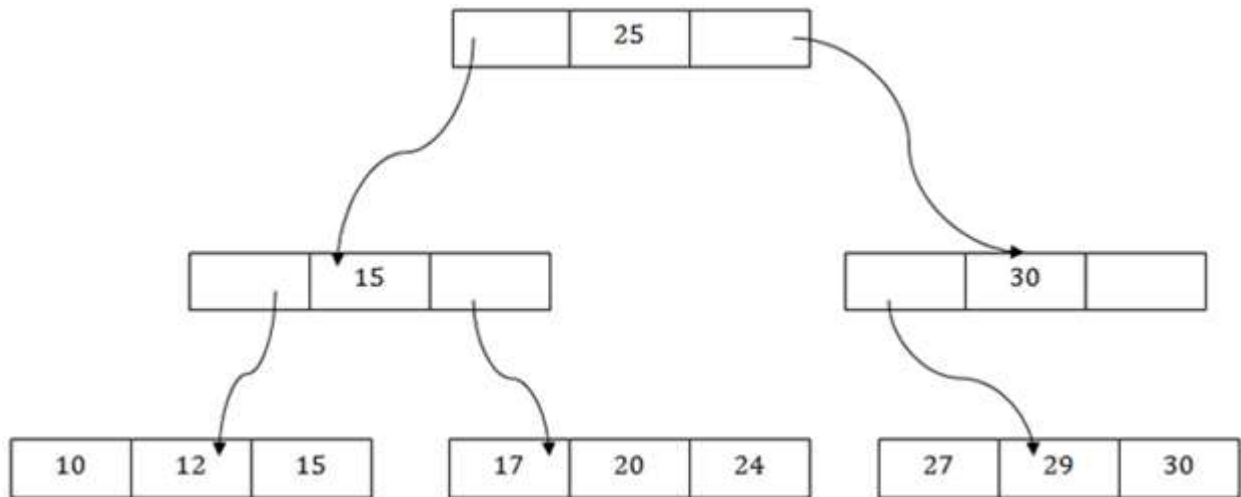


Fig: B+ tree

Pros of B+ file Organization:

- ✚ Very fast **searching, insertion, and deletion** due to balanced tree structure
- ✚ Efficient for both **exact match and range queries** (e.g., 10–50)
- ✚ Data is stored in sorted order, making sequential access easy
- ✚ Tree remains balanced, so performance stays stable even with large data

Cons of B+ file Organization:

- ✚ Structure is complex and harder to implement and manage
- ✚ Requires extra storage for index nodes (memory overhead)
- ✚ Insertions and deletions are costly because the tree must be rebalanced
- ✚ Not efficient for small databases due to extra overhead

Fixed Length Records:

It is a type of records in a file storage with fixed predefined size, meaning each record occupies an exactly same amount of space regardless of the actual data stored inside it. They contain field with Char datatype where each field is assigned a fixed number of bytes. If the data is smaller, the remaining space stays unused, and if the data is larger than the allocated record size, it must be trimmed or cannot be stored fully. This uniform structure makes processing and accessing records faster and simpler, but it can also lead to wasted storage space due to unused allocated memory.

For example, if a student record is fixed to 50 bytes, then every student entry (name, roll number, class, etc.) must fit within those 50 bytes; even if one student's data uses only 30 bytes, the remaining 20 bytes will still be reserved and unused.

Variable Length Records:

It is a type of record whose size is not fixed and varies based on the data stored in it. They may include fields like varchar, arrays, or repeating values, where different records can have different lengths. To manage them, a record is split into a fixed part (for numbers, dates, etc.) and a variable part, where variable fields are stored using an (offset, length) pair to locate data easily.

For example, a student record with Address = "Kathmandu" will be smaller than another with Address = "Kathmandu, Nepal".

Data Dictionary Storage:

Data Dictionary Storage is a special part of a database that stores metadata (data about data) instead of actual user data. It contains information such as table names, column (attribute) names, data types and sizes, views, and integrity constraints. It works like a reference guide that tells the database system how all data is structured and related.

The importance of a data dictionary is that it helps the DBMS manage and control the entire database efficiently. It allows easy access to information about tables and their structure, ensures consistency and integrity of data, helps in query processing, and supports database administration by making it easier to update and maintain the database system.

Unit 2

Indexing and Hashing

Indexing:

Indexing is a technique used in databases to improve the speed of data retrieval by reducing the number of disk accesses during query processing. It creates a special data structure that helps the system quickly locate required records without scanning the entire table. Indexes are usually created on one or more columns of a table to make searching more efficient.



Fig: Structure of Index in database

In an index structure, the first column is the search key, which contains a sorted copy of the primary key or candidate key of the table. This sorting allows fast searching of values. The second column contains data references, which are pointers to the disk locations where the actual records are stored. By using these pointers, the database can directly access the required data, making retrieval much faster and more efficient.

The need / benefits / advantages of Indexing is given as:

- ✚ Speeds up data retrieval
- ✚ Reduces disk I/O operations
- ✚ Avoids full table scan
- ✚ Improves query performance
- ✚ Helps in faster searching and filtering
- ✚ Efficient for large databases
- ✚ Enhances performance of SELECT, WHERE, and ORDER BY queries

The disadvantages of Indexing is given as:

- ✚ Requires extra storage space
- ✚ Slows down INSERT, UPDATE, and DELETE operations
- ✚ Not useful for small tables
- ✚ Can become inefficient if not designed properly
- ✚ Multiple indexes may reduce overall performance

Factors for Evaluating Indices:

Factors for Evaluating Indices refer to the criteria's used to measure efficiency of an index in a database. They help us understand whether an index improves performance or creates extra overhead. We evaluate indices based on following factors:

- ✚ **Access types:** Types of queries the index can support efficiently (exact match, range search).
- ✚ **Access time:** Time taken to locate and retrieve data.
- ✚ **Insertion time:** Time required to add new records and update the index.
- ✚ **Deletion time:** Time required to remove records and maintain the index.
- ✚ **Space overhead:** Extra storage needed to store the index structure.

Types of Indices:

The different types of Indices are as follows:

1. Ordered Indices
2. Primary Index (Dense and Sparse)
3. Secondary Index
4. Multilevel Index
5. B+ Tree Index

Ordered Index:

Ordered indices are indexes in which the data entries are stored in a sorted order (usually based on the key value). This sorting helps in faster searching compared to scanning the entire table.

Example:

Suppose we have an employee table with thousands of records, each record being 10 bytes long, and employee IDs are in order (1, 2, 3, ...). If we need to find ID 543:

- 🚦 **Without index:** The DBMS must scan records one by one from the beginning, which takes a lot of time and disk access.
- 🚦 **With ordered index:** The DBMS first searches the index (which is much smaller) and directly points to the location of record 543, requiring far fewer disk reads.

So, ordered indices significantly reduce search time by avoiding full table scanning.

Primary Index:

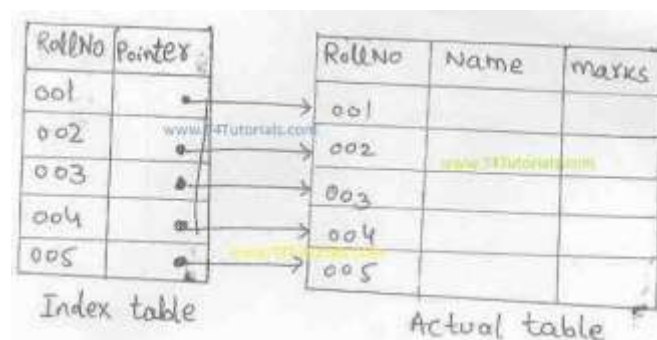
A primary index is an index that is created on the primary key of a table. Since the primary key uniquely identifies each record, the index helps in quickly locating a specific record in the database. In a primary index, the primary key values are stored in sorted order, which makes searching efficient and faster compared to scanning the entire table. This improves the overall performance of data retrieval operations. Primary index can be classified into 2 types. They are:

1. Dense Primary Index
2. Sparse Primary Index

Dense Primary Index:

A dense index is an index structure in which there is an index entry for every record in the data file. This means each search key value in the table has a corresponding entry in the index, which directly points to the location of the actual record on disk.

Because it contains entries for all records, the number of index records is equal to the number of records in the main table. This makes searching very fast since any record can be directly located through the index. However, it requires more storage space because a separate index entry is maintained for every record.



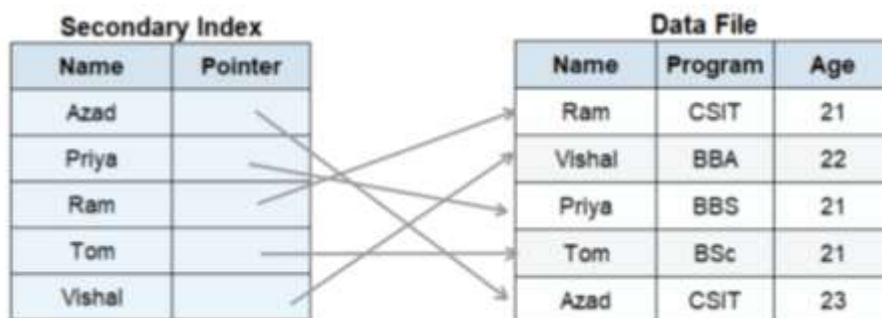
Sparse Primary Index:

A sparse index is an index in which index entries are created only for some records in the data file, not for every record. Instead of pointing to each individual record, each index entry points to a block (group of records) in the main table. In this type of index, the search key values are not stored for all records, so it uses less space compared to a dense index. However, it is slightly slower to search because after reaching the correct block using the index, the system may need to scan within that block to find the exact record.



Secondary Index:

A secondary index is an index created on any non-primary key column to improve performance of data retrieval. It is used when we need to search records using a field other than the primary key. In this type of indexing, the index stores the values of the selected field along with pointers to the actual records in the table. Since this field may have duplicate or unique values, the index helps quickly locate all matching records without scanning the whole table.



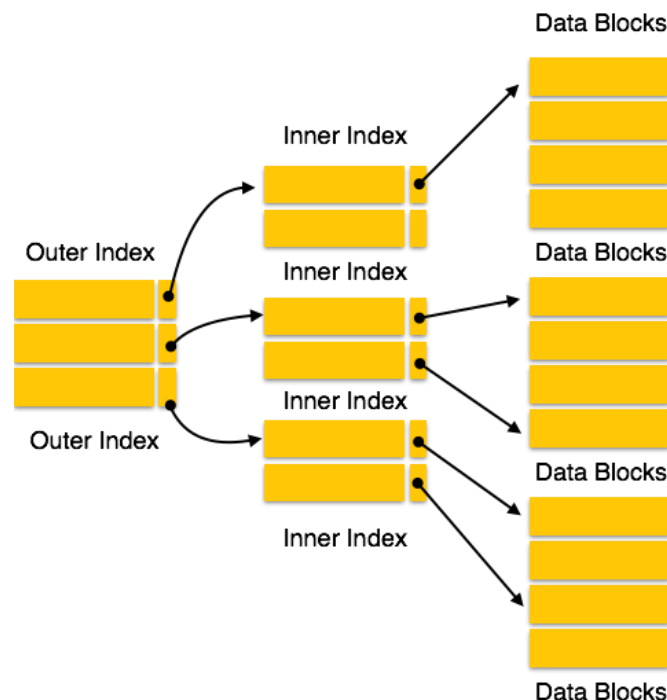
Example:

Consider the student table shown in the figure. A secondary index is created on the Name column to improve searching performance. The index table stores the name values along with pointers to the corresponding records in the data file. For example, when searching for “Priya,” the database uses the index to directly locate the required record instead of scanning the entire table, making data retrieval faster and more efficient.

Multilevel Index:

A multilevel index is an indexing method where a large index is divided into several smaller index levels to make searching more efficient. When the database grows, a single-level index becomes very large and cannot fit in main memory, which leads to slow searching due to multiple disk accesses. To solve this, the index is broken into multiple levels. The top-level index is very small and can easily fit into memory. This top level points to the lower-level indexes, and those lower levels finally point to the actual data records.

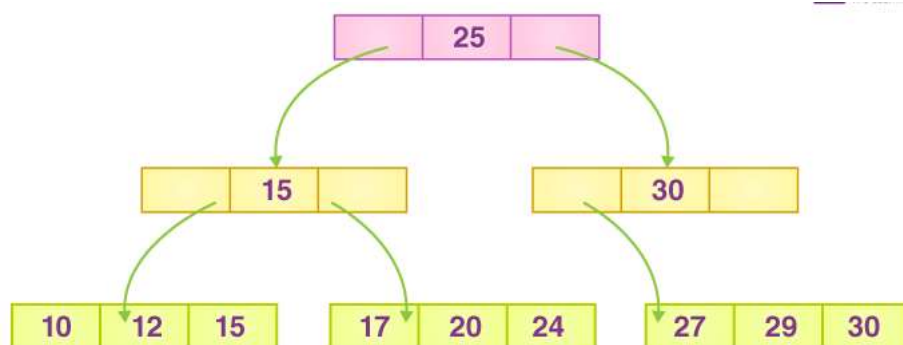
In simple words, multilevel indexing works like a hierarchy of indexes, where each level helps quickly narrow down the search until the required data is found. This reduces disk access and improves search speed.



B+ Tree Index:

A B+ Tree index is a balanced tree structure used in databases to store and access data efficiently. It improves search performance by organizing data in multiple levels. In a B+ Tree, internal nodes store only key values for navigation, while leaf nodes store the actual data pointers. All leaf nodes are kept at the same level, which maintains balance and ensures uniform search time.

The leaf nodes are linked together, allowing fast sequential access in sorted order. To find data, the search begins at the root and moves down through internal nodes until it reaches the correct leaf node. Overall, B+ Tree indexing supports fast searching, insertion, deletion, and efficient range queries, making it highly suitable for large databases.



Example:

The given B+ Tree index shows how data is stored in a balanced structure for efficient searching. The root node contains key 25, which divides the data into two parts, directing smaller values to the left and larger values to the right. Internal nodes like 15 and 30 further guide the search process, while the leaf nodes store the actual data values in sorted order (10, 12, 15, 17, 20, 24, 27, 29, 30).

All leaf nodes are linked together, allowing fast sequential access. Overall, this structure ensures quick and efficient searching, insertion, deletion, and range queries.

Hashing:

Hashing is a technique used in large databases to efficiently store and retrieve records without searching through index structures. It works by applying a hash function to a key value, which directly computes the address where the record should be stored on disk. During insertion, the hash function generates the storage location and the record is placed directly into a data bucket or data block at that address. During retrieval, the same hash function is applied to the key to quickly locate the required record. These storage locations are called data buckets or data blocks, where records are organized based on their computed hash values.

Hashing techniques are mainly classified into two types.

1. Static Hashing
2. Dynamic Hashing

Hash function:

A hash function in hashing is used to compute the address of a data record in a database. It takes a column value (usually the primary key) as input and generates the storage location of the corresponding data block. In most cases, the primary key is used because it uniquely identifies each record.

A hash function can be a simple or complex mathematical function depending on the system. In some cases, the primary key itself can directly represent the address of the data block, meaning each record is stored at the location derived from its key value.

Static Hashing:

Static hashing is a technique in which the number of storage buckets is fixed beforehand and does not change over time. A hash function is used to convert a search key into a bucket address, and the same key will always produce the same address. For example, if a 'mod-4' hash function is used, it generates a fixed set of possible bucket addresses, and each record is mapped to one of these buckets. Since the number of buckets is fixed, the structure remains constant even as data increases.

Operations in Static Hashing:

- ✚ **Insertion:** The hash function $h(K)$ computes the bucket address for the key K , and the record is stored in that bucket.
- ✚ **Search:** The same hash function is applied to the search key to directly locate the required bucket and retrieve the record.
- ✚ **Delete:** The record is first found using the hash function and then removed from its bucket.

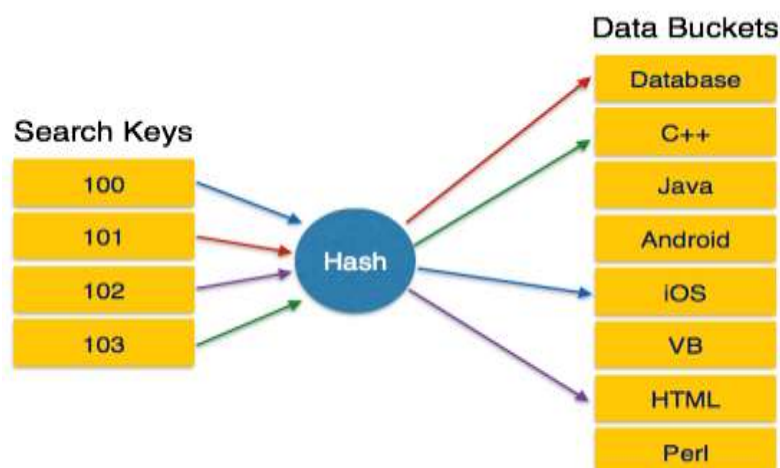


Fig: Static Hashing

Dynamic Hashing:

[or, Extended Hashing]

Dynamic hashing is a hashing technique designed to overcome the limitation of static hashing, where the number of buckets remains fixed. In dynamic hashing, the database can grow or shrink, and the number of buckets is adjusted accordingly by adding or removing them as needed. Because of this flexibility, it is also called extended hashing. The hash function is designed to generate a large range of values, but only a portion of those values is used initially for addressing buckets.

Organization of Dynamic Hashing:

In dynamic hashing, only a prefix (a few leading bits) of the hash value is used as the hash index. Each index has a depth value that indicates how many bits are currently used to determine the bucket address. Based on these bits, up to 2^n buckets can be addressed. When buckets become full, the depth is increased, allowing more bits of the hash value to be used and doubling the number of buckets.

Operations in Dynamic Hashing:

-  **Query/Search:** The system checks the depth value and uses the required number of bits from the hash value to locate the correct bucket.
-  **Insertion:** The hash value is computed and the record is inserted into the corresponding bucket. If the bucket is full, new buckets are created, the depth is increased, and the hash structure is reorganized.
-  **Update:** First locate the record using the hash function and then modify it.
-  **Deletion:** Locate the record using the hash function and remove it from the bucket.

Note:

Hashing is not suitable for range queries because it does not maintain order; it works best when data is random and evenly distributed.

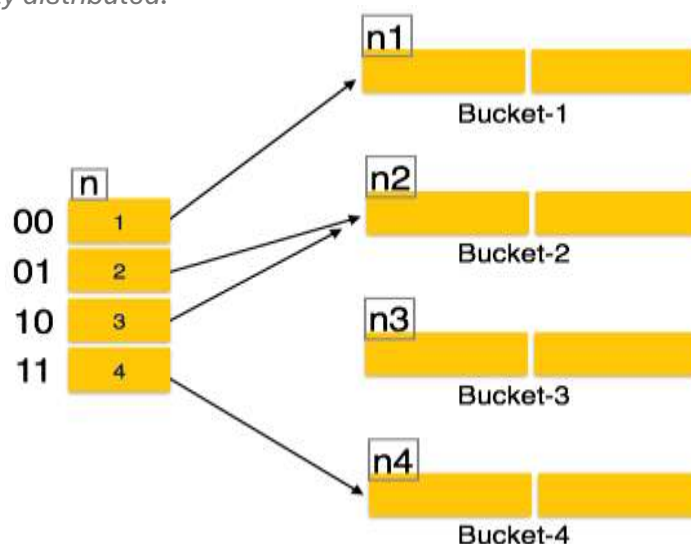


Fig: Dynamic Hashing

Bucket Overflow Handling:

Bucket overflow occurs when a hash function maps more records to a bucket than it can store. This situation is called a collision. It is a major limitation in static hashing because the number of buckets is fixed. To handle collisions, following techniques are used.

1. Overflow chaining (Closed Hashing)
2. Linear Probing (Open Hashing)

Overflow Chaining (Closed Hashing):

When a bucket becomes full, an additional bucket is created for the same hash value and linked to the original bucket. This forms a chain of buckets, allowing multiple records with the same hash address to be stored together.

Example:

If $h(k) = 1$ and the bucket is already full, a new bucket is created and linked to bucket 1.

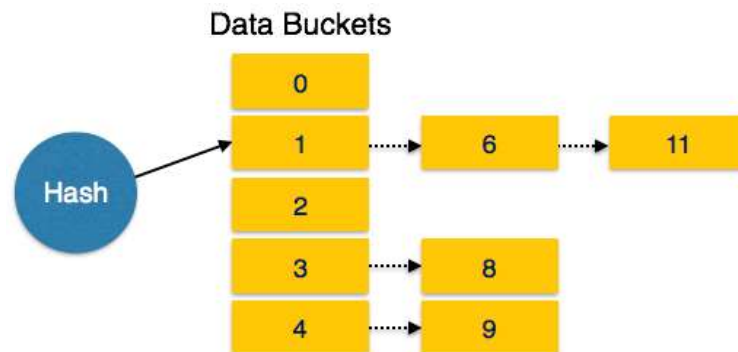


Fig: Overflow Chaining

Linear Probing (Open Hashing):

When a bucket is already occupied, the system searches sequentially for the next available empty bucket and stores the record there. Instead of linking new buckets, the record is placed in the nearest free location.

Example:

If $h(k) = 1$ and the bucket is full, the record is placed in the **next available free bucket** ($1 \rightarrow 2 \rightarrow 3 \dots$).

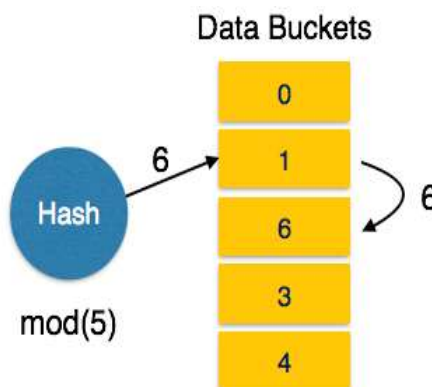


Fig: Linear Probing

Hash Indices:

A hash index is a secondary index structure that stores search keys along with pointers to their corresponding data records using a hash-based organization. A hash function is applied to the search key to determine the bucket where the key and its pointer are stored, and if a bucket becomes full, overflow buckets are used. In practice, hash indices are mainly used as secondary indexes because if the data file is already organized using hashing, a separate hash index is usually not required.

Index Definition in SQL:

1. CREATE INDEX statement

Used to create an index on one or more columns. It allows duplicate values.

Syntax	Example
<pre>CREATE INDEX index_name ON table_name (column1, column2, ...);</pre>	<pre>CREATE INDEX idx_name ON Person (lastName, firstName);</pre>

2. UNIQUE INDEX statement

Used to create an index that does not allow duplicate values.

Syntax	Example
<pre>CREATE UNIQUE INDEX index_name ON table_name (column1, column2, ...);</pre>	<pre>CREATE INDEX website_idx ON website (siteName);</pre>

3. DROP INDEX statement

Used to delete an existing index from a table.

Syntax	Example
<pre>DROP INDEX index_name;</pre>	<pre>DROP INDEX websites_idx;</pre>

Unit 3

Query Processing

Query Processing:

Query Processing is the process of converting high-level database queries, such as SQL queries, into low-level operations that the database system can execute to retrieve data. It involves several steps, including translating the query, optimizing it for better performance, and executing it at the physical file system level to produce the required result. Query processing uses concepts of relational algebra and file structures to efficiently access and manage data. In simple terms, it explains how a database understands, processes, and optimizes queries to return results quickly and accurately.

Steps for Query Processing:

1. Parsing and Translation
2. Query Optimization
3. Query Evaluation and Execution

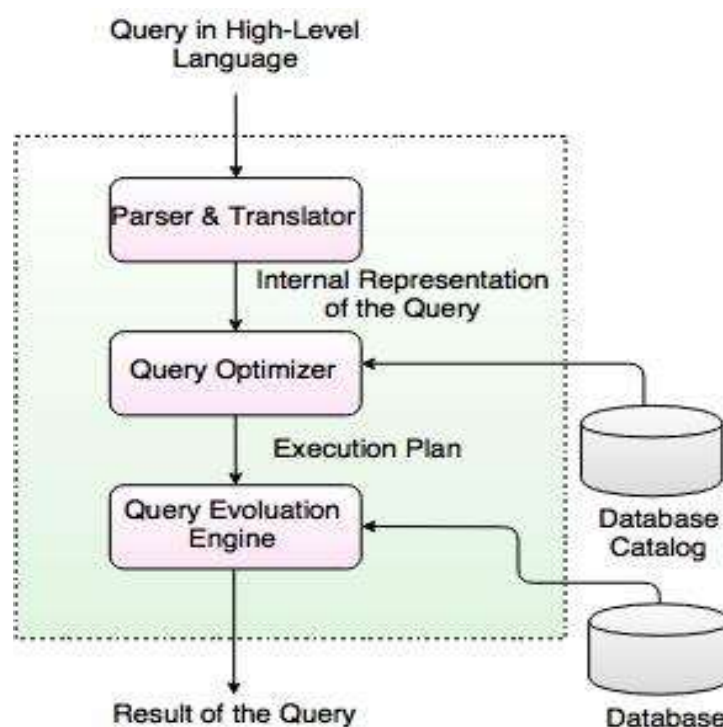


Fig. Query Processing

Parsing and Translation:

The first step in query processing is parsing and translation. The user writes a query in a high-level language such as SQL, and this query is given to the parser and translator. The parser checks the query for syntax and semantic errors, such as incorrect SQL keywords, missing tables, or invalid column names. In this stage, the system performs either a hard parse or soft parse depending on whether a matching query already exists in memory. If the query is correct, the translator converts the SQL query into an internal form, usually relational algebra expressions or a query tree, which the database system can understand and process further.

Example:

SQL Query (INPUT): `SELECT name FROM Student WHERE age > 20;`

Then, parser checks whether the SQL query is written correctly or not. It verifies all keywords and checks if the table and column exists or not. After verification, the translator converts the Query into internal representation such as relation algebra as:

$$\pi_{name}(\sigma_{age>20}(Student))$$

Where,

 σ means **selection** (choose rows where age > 20)

 π means **projection** (display only the name column)

Query Optimization:

After translation, the query is sent to the query optimizer. The optimizer's job is to find the most efficient way to execute the query. It receives information from the database catalog, which stores metadata such as table size, indexes, file structure, and statistics about the data. Using this information, the optimizer compares different execution methods and selects the best one with minimum cost, reduced memory usage, and faster execution time. The selected method is called the execution plan.

Example:

For the query `SELECT name FROM Student WHERE age > 20;`, the query optimizer checks the database catalog to find the best execution method. If an index exists on the age column, the optimizer may choose to use the index to quickly locate records where age is greater than 20 instead of scanning the entire table. This selected efficient method is called the execution plan.

Query Evaluation and Execution:

In the final step, the execution plan is passed to the query evaluation engine. This component actually performs the operations needed to execute the query. It accesses the database files, retrieves the required data, performs operations like selection, projection, joining, and sorting, and generates the final output. After execution is completed, the query result is returned to the user.

Example:

After optimization, the query evaluation engine executes the execution plan. It accesses the Student table, retrieves all records where age > 20, extracts only the name column, and returns the final result to the user. For example, the output may display names like Ram, Sita, and Hari.

What do you mean by Hard Parse?

A hard parse happens when the database receives a SQL query that is completely new or cannot be reused from memory. In this case, the database must go through the full query processing cycle again, including parsing, optimization, and generating a new execution plan. This process is time-consuming because the optimizer has to analyze the query from scratch and check the database catalog again. Hard parsing usually happens when a query is executed for the first time or when there is no matching cached version available.

What do you mean by soft Parse?

A soft parse happens when a similar or identical SQL query has already been executed before and its execution plan is already stored in the database's memory (shared pool or cache). Instead of redoing all steps, the database simply reuses the existing execution plan and directly executes the query. This makes soft parsing much faster and more efficient because it avoids repeated optimization work and reduces processing overhead.

Measure of Query Cost:

The measure of query cost refers to the method used in a database system to estimate how expensive it is to execute a query. In simple terms, it helps the DBMS decide which query execution plan is the most efficient by calculating the resources needed to run the query, such as time and system usage.

The main measures used to estimate query cost are:

1. Disk Access (I/O Cost):

Number of disk operations needed to read/write data. It includes block transfers (moving data between disk and memory) and disk seeks (positioning the disk head). This is the most important cost because disk access is slow.

2. CPU Cost:

Time taken by the processor to perform operations like sorting, joining, and filtering data.

3. Communication Cost:

Cost involved in transferring data in distributed or parallel database systems.

4. Response Time: Total time taken to execute the query and produce the result.

Disk access is usually given the highest importance because it is much slower compared to CPU and other operations.

Example:

Consider the query:

```
SELECT name FROM Student WHERE age > 20;
```

[Reading means fetching data from disk to Main Memory]

 **Disk Access (I/O cost):** If the Student table has 10,000 records stored in 1,000 disk blocks, and the system performs a full table scan, then all 1,000 blocks may need to be read from disk. This gives the main cost of the query.

 **CPU Cost:** After reading data into memory, the CPU checks each record to see whether age > 20 and extracts the name column. This processing time is relatively small compared to disk access.

 **Communication Cost:** If this query is executed in a distributed database, the matching records may need to be sent from one server to another, adding communication cost.

 **Response Time:** The total time taken includes reading disk blocks, filtering data, and returning the final result to the user.

Selection Operation:

The selection operation is used to retrieve specific records from a table based on a condition (for example, WHERE age > 20). It is usually performed using a file scan, which simply reads the data file block by block to find matching records. This is the most basic and lowest-level method of accessing data in query processing.

Selection using Index (Index Scan):

Instead of scanning the whole file, a database can use an index, which works like a shortcut to quickly locate required records. These indexes are also called access paths, because they provide a direct path to the data.

The types of Index-based Selection are:

1. Primary Index (Equality on key)
2. Primary Index (Equality on non-key)
3. Secondary Index (Equality Key or non-key)

Primary Index (Equality on key):

This is used when the condition is on a primary key field. Since the data is already ordered by this key, the index directly finds the required record.

Example:

```
SELECT * FROM Student WHERE roll = 101;
```

Here, only one record is found because roll is unique.

Primary Index (Equality on non-key):

Here, the condition is on a non-key field, so multiple records can match. The index helps locate the records, but more than one block may need to be searched.

Example:

```
SELECT * FROM Student WHERE department = 'CS';
```

Here, many students can belong to CS, so multiple records are returned.

Secondary Index (Equality on key or non-key):

This index is created on a field that is not used for ordering the file. It can be used for both unique and non-unique fields. If the value is unique, one record is returned; if not, multiple records are fetched, possibly from different disk blocks.

Example:

```
SELECT * FROM Student WHERE email = 'abc@gmail.com';
```

-- one record

```
SELECT * FROM Student WHERE city = 'Kathmandu';
```

-- many records

Selection Operations with Comparisons:

Selection with comparisons means retrieving records using conditions like $<$, $>$, $\leq$, $\geq$ instead of exact matching ($=$). This can be done using a file scan or indexes. There are 2 types. They are:

1. Primary Index (Comparison):

When a comparison condition is applied on a field with a primary index (usually primary key), the primary index helps directly jump to the starting point of the required range and then reads all matching records sequentially. This is because the records are already stored in sorted order based on the primary key, making range searching efficient.

Example:

```
SELECT * FROM Student WHERE age > 20;
```

The index quickly locates the first value greater than 20, then all matching records are read one by one.

2. Secondary Index (Comparison):

When a comparison condition is applied on a non-ordering (non-primary) field, a secondary index is used to find matching values. The index first locates the required entries and then accesses the actual records using pointers. Since the data is not stored in sorted order for this field, the matching records are scattered across different disk blocks. Therefore, all matched records are accessed through pointers, which may require multiple disk accesses and makes it slower than a primary index.

Example:

```
SELECT * FROM Student WHERE age >= 18;
```

The secondary index finds all records where $\text{age} \geq 18$ using index entries, but each matching record is accessed through pointers, and since records are scattered across different disk blocks, multiple disk reads are required to fetch all results.

Implementation of Complex Selection:

Complex selection involves retrieving records using multiple conditions combined with logical operators such as AND ($\wedge$), OR ($\vee$), and NOT ($\neg$).



AND ($\wedge$):

Both conditions must be true. The system first applies one condition and then filters the result using the second condition.



OR ($\vee$):

Either condition can be true. The system retrieves records satisfying any one condition and then combines the results.



NOT ($\neg$):

Excludes records that satisfy the given condition.

The evaluation follows operator precedence, where NOT has the highest priority, followed by AND, and then OR. If operators have the same precedence, evaluation is done from left to right (associativity).

Example:

```
SELECT *  
FROM Student  
WHERE age > 18 AND city = 'Kathmandu' OR department = 'CS';
```

Join Operation:

A join operation is used to combine rows from two or more tables based on the related column between them. It is widely used to retrieve related data stored in different tables.

The join algorithms / Join processing Methods / Join Methods are given as:

1. Nested Loop Join
2. Block Nested Loop Join
3. Indexed Nested Loop Join

Nested Loop Join:

A Nested Loop Join is the simplest join method used in query processing. It works by comparing each tuple (row) of one table with every tuple of another table to find matching records based on the join condition. It is called “nested” because one loop is inside another loop.

Pseudo code:

```
for each tuple t1 in Table1 {  
    for each tuple t2 in Table2 {  
        if (join condition between t1 and t2 is true) {  
            add tuple (t1, t2) to result  
        }  
    }  
}
```

Explanation:

In this pseudo code, the system takes one tuple from Table1 and compares it with all tuples in Table2 one by one. If the join condition is satisfied, the two tuples are combined and added to the result. This process is repeated for all tuples in Table1 to produce the final joined output.

Block Nested Loop Join:

A Block Nested Loop Join is an improved version of the nested loop join. Instead of taking one tuple at a time, it takes a block (group of tuples) from the first table and compares it with all tuples of the second table. This reduces the number of repeated scans and improves performance compared to the simple nested loop join.

Pseudo code:

```
for each block B1 in Table1 {
  for each tuple t2 in Table2 {
    for each tuple t1 in B1 {
      if (join condition between t1 and t2 is true) {
        add tuple (t1, t2) to result
      }
    }
  }
}
```

Explanation:

In this pseudo code, the system takes a block (group of tuples) from Table1 instead of a single tuple. For each block, it scans all tuples of Table2 and compares them with every tuple inside the block. If the join condition is satisfied, the matching tuples are combined and added to the result. This reduces the number of disk accesses and makes the process faster than the basic nested loop join.

Indexed Nested Loop Join:

An Index Nested Loop Join is a join method where an index is used on the second table (inner table) to quickly find matching records instead of scanning the entire table. For each tuple in the first table, the system uses the index to directly locate matching tuples in the second table, making it faster than normal nested loop join.

Pseudo Code:

```
for each tuple t1 in Table1 {
  use index on Table2 to find matching tuples t2
  for each tuple t2 found{
    if (join condition between t1 and t2 is true) {
      add tuple (t1, t2) to result
    }
  }
}
```

Explanation:

In this method, the system takes one tuple from Table1 and uses an index on Table2 to directly find matching tuples instead of scanning all records. Only the matching tuples are checked against the join condition, and if it is satisfied, the tuples are combined and added to the result. This makes the process much faster compared to simple nested loop join, especially when a good index exists on the second table.

Duplicate Elimination Operation:

Duplicate elimination is an operation used in query processing to remove repeated tuples (rows) from the result of a query. It is mainly used when the DISTINCT is applied. The DBMS scans the result set and keeps only one copy of each identical tuple. This can be implemented using sorting (then removing duplicates) or hashing techniques. It ensures that the final output contains only unique records, since relational algebra does not allow duplicate tuples.

Example:

```
SELECT DISTINCT city FROM Student;
```

This query returns only unique city names by removing duplicate entries from the result.

Projection:

Projection is used to select only specific columns (attributes) from a table and ignore the remaining columns. It is represented by π (pi) in relational algebra. The DBMS reads each tuple and keeps only the required attributes, removing the rest.

Example:

```
SELECT name, age FROM Student;
```

This query returns only the name and age columns from the Student table and ignores all other columns.

Set Operations:

Set operations in DBMS are used to combine or compare results of two SELECT queries. They work only when both queries have the same number of columns and compatible data types. The set operations are of following types:

1. UNION
2. INTERSECT
3. EXCEPT (Minus)

UNION:

UNION combines the results of two queries and removes duplicates. Given SQL command returns all unique cities from both Student and Teacher tables.

```
SELECT city FROM Student
UNION
SELECT city FROM Teacher;
```

INTERSECT:

INTERSECT returns only the common records present in both queries. Given SQL command returns cities that are present in both Student and Teacher tables.

```
SELECT city FROM Student
INTERSECT
SELECT city FROM Teacher;
```

EXCEPT (Minus):

EXCEPT returns records that are in the first query but not in the second. Given SQL command returns cities that are in Student but not in Teacher.

```
SELECT city FROM Student
EXCEPT
SELECT city FROM Teacher;
```

JOIN:

A join is an operation used in DBMS to combine rows from two or more tables based on a related column between them. It helps retrieve related data that is stored in different tables. The types of join are explained below:

1. **INNER JOIN:** Returns only the records that have matching values in both tables.
2. **LEFT JOIN:** Returns all records from the left table and matched records from the right table. If no match is found, NULL is returned.
3. **RIGHT JOIN:** Returns all records from the right table and matched records from the left table.
4. **FULL OUTER JOIN:** Returns all records from both tables, matching where possible and filling NULL where no match exists.

Example:

```
SELECT Student.name, Department.dept_name  
FROM Student  
INNER JOIN Department  
ON Student.dept_id = Department.dept_id;
```

This shows only students who have a matching department.

Aggregation:

Aggregation is a process used in DBMS to perform calculations on a set of values and return a single summarized value. It is used to analyze data in a table. These functions work on a group of rows and are often used with the GROUP BY clause. Some common aggregate functions are:

-  **COUNT()** → counts number of rows
-  **SUM()** → adds all values
-  **AVG()** → finds average value
-  **MIN()** → finds smallest value
-  **MAX()** → finds largest value

Example:

```
SELECT COUNT(*) FROM Student;
```

This returns the total number of students in the table.

Materialized Evaluation:

Materialized evaluation is a query processing method where the result of each operation is fully computed and stored (in memory or disk) before moving to the next operation. These stored intermediate results are then used as input for the next step in the query.

For example:

```
SELECT name  
FROM Student  
WHERE age > 20;
```

In above materialized evaluation example, the DBMS first executes the selection operation `age > 20` completely and stores the intermediate result (all students older than 20) in memory or disk. After that, it applies the projection operation to extract only the name column from that stored result.

Pipelining Evaluation:

Pipelining evaluation is a query processing method where the DBMS does not store intermediate results. Instead, it directly passes the output of one operation as input to the next operation. In this approach, each tuple (row) is processed step by step and immediately sent to the next operator, which makes execution faster and reduces memory usage compared to materialized evaluation.

Example:

```
SELECT name
FROM Student
WHERE age > 20;
```

In pipelining, as soon as a tuple satisfies the condition $age > 20$, it is immediately passed to the projection step to extract name. There is no need to store the intermediate result in memory or disk.

Difference Between:

Materialized Evaluation	Pipelining Evaluation
❖ One operation finishes completely before the next starts.	❖ One operation finishes completely before the next starts.
❖ Full result of each step is stored temporarily.	❖ Results are passed directly to the next step.
❖ Needs extra memory or disk space.	❖ Uses very little extra storage.
❖ Slower because each step must finish completely first.	❖ Faster because multiple steps work together.
❖ Example: First complete selection, then join.	❖ Example: Selection output goes directly into join.

Unit 4

Query Optimization

Query Optimization:

Query optimization is the process of selecting the most efficient way to execute a SQL statement. Since SQL is a non-procedural language, the database optimizer has the flexibility to merge, rearrange, and execute operations in different orders to improve performance. It uses statistics about the data, such as table size and data distribution, to decide the best execution strategy for each query, ensuring faster and more efficient retrieval of results.

The importance of Query Optimization:

- 🚦 Improves the speed of query execution and data retrieval.
- 🚦 Reduces the cost and time required to process queries.
- 🚦 Minimizes unnecessary load on the database system.
- 🚦 Enhances the overall performance and efficiency of the DBMS.
- 🚦 Optimizes the use of memory and other system resources.

Estimating Statics of Expression Result:

To select the most efficient query execution plan, the query optimizer uses statistical information stored in the data dictionary. This information is collected when tables are created and updated during DDL and DML operations. It helps the optimizer estimate the cost of different query plans.

The main statics used are:

- 🚦 **Number of records:** Total rows in a table. It helps decide the execution order, often processing smaller tables first to reduce intermediate results.
- 🚦 **Number of records per block:** Average number of records stored in one disk block, used to estimate storage efficiency.
- 🚦 **Number of blocks:** Total disk blocks used by a table. It helps estimate the number of disk accesses required.
- 🚦 **Record length:** Size of each record. It is important for tables with variable-length records as it affects storage and access cost.
- 🚦 **Number of distinct values:** Number of unique values in a column. It is useful for estimating the result size of selection and aggregation operations.
- 🚦 **Index levels:** Indicates whether the index is single-level or multi-level (like B+ tree). It helps estimate the number of block accesses needed.
- 🚦 **Selection cardinality:** Average number of records matching a value of attribute A. It helps estimate how many rows will satisfy a condition.

Catalog Information:

Catalog information refers to the metadata (data about data) stored in the database system. It is maintained in the data dictionary (system catalog) and contains details about database objects and their properties. The query optimizer uses this information to choose the most efficient query execution plan.

Catalog information includes:

- 🚦 Table names and column names

- ✚ Data types and sizes of attributes
- ✚ Number of records and blocks in tables
- ✚ Index information
- ✚ Constraints and keys
- ✚ Statistics about data distribution

Selection Size Estimation:

Selection size estimation is used to find the approximate number of records that will satisfy a selection condition. It helps the DBMS decide the best query execution plan.

- ✚ **For Equality Condition Like:** SELECT * FROM Table WHERE column = value;
If attribute A is a key attribute, then the estimated result size is 1, because a key column contains unique values and only one record can match.

- ✚ **For range or comparison conditions like:** SELECT * FROM Table WHERE A ≤ value;
Here, the number of rows satisfying selection condition is estimated using given formula:

$$n_r \times \frac{\text{value} - \min(A)}{\max(A) - \min(A)}$$

If value > min(A),
Otherwise the estimate is 0.

- n_r = total number of records in the table
- $\min(A)$ = minimum value of attribute A
- $\max(A)$ = maximum value of attribute A

Example:

Suppose:

Total records = 100, minimum age = 18, maximum age = 25.

Query: SELECT * FROM Student WHERE age ≤ 20;

Estimated Size: $100 \times \frac{20 - 18}{25 - 18} \approx 29$

So, approximately 29 records are expected to satisfy the condition age ≤ 20.

Estimation of Join Size:

Join size estimation is used to estimate the approximate number of records that will be produced after performing a join operation. The query optimizer uses this estimation to choose the most efficient join method and execution plan. Different types of joins use different formulas for estimating the result size. They are explained below:

1. Cartesian Product (Cross Join):

In a Cartesian product, every tuple of one table is combined with every tuple of the other table.

Formula: $n_r \times n_s$

n_r = number of records in table R

n_s = number of records in table S

2. Inner Join / Natural Join (Both the join use same formula):

For inner join and natural join, estimation is based on the distinct values of the joining attributes.

Formula:

$$\frac{n_r \times n_s}{\max(d(A), d(B))}$$

n_r = number of records in table R

n_s = number of records in table S

$d(A)$ = number of distinct values of attribute A in R

$d(B)$ = number of distinct values of attribute B in S

This formula estimates the number of matching tuples between the two tables.

3. Left Outer Join

A left outer join includes all tuples from the left table, even if no matching tuples exist in the right table.

Formula: $\text{Result size} \geq n_r$

Result size will always be at least the number of tuples in the left table.

4. Right Outer Join

A right outer join includes all tuples from the right table, even if no matching tuples exist in the left table.

Formula: $\text{Result size} \geq n_s$

Result size will always be at least the number of tuples in the right table.

5. Full Outer Join:

A full outer join includes all tuples from both tables, whether matching or not.

Formula: $\text{Result size} \geq \max(n_r, n_s)$

Result size will always be at least the size of the larger table.

Transformation of Relational Expressions:

It is the process of rewriting a query expression into an equivalent form that produces the same result but is more efficient to execute. It is mainly used in query optimization to reduce cost and improve performance. The DBMS uses different transformation rules to convert relational algebra expressions without changing their meaning.

Transformation Rules:

Transformation rules are used to rewrite relational algebra expressions into equivalent forms without changing the final result. These rules help the query optimizer choose a more efficient execution plan. The rules are:

1. **Selection Rules:** Selection conditions can be applied in any order.

Cascade rule: $\sigma_{C1 \wedge C2}(R) = \sigma_{C1}(\sigma_{C2}(R))$

Commutative rule: $\sigma_{C1}(\sigma_{C2}(R)) = \sigma_{C2}(\sigma_{C1}(R))$

2. **Projection Rules:** Only final required attributes are needed.

Cascade rule: $\pi_{A1}(\pi_{A2}(R)) = \pi_{A1}(R)$

3. **Join Commutative Rule:** Order of join can be changed.

$$R \bowtie S = S \bowtie R$$

4. **Join Associative Rule:** Grouping of joins can be changed.

$$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$$

5. **Selection Push-down Rule:** Apply selection before join to reduce data size.

$$\sigma(R \bowtie S) = (\sigma R) \bowtie S$$

Examples of Transformation of Relational Expressions:

Below are simple examples showing how a query can be rewritten into an equivalent but more efficient form.

1. Selection Push-Down

Original Expression: $\sigma_{age>20}(Student \bowtie Department)$

Transformed Form: $(\sigma_{age>20}(Student)) \bowtie Department$

Here, we filter Student first, then join → reduces number of rows before join.

2. Projection Push-Down

Original Expression: $\pi_{name,dept}(Student \bowtie Department)$

Transformed Form: $(\pi_{name,dept}(Student)) \bowtie (\pi_{dept}(Department))$

Here, only required columns are kept early → reduces data size.

3. Join Commutativity

Original Expression: $Student \bowtie Department$

Transformed Form: $Department \bowtie Student$

Here, Order of joining tables is changed, result remains same.

4. Join Associativity

Original Expression: $(Student \bowtie Department) \bowtie Course$

Transformed Form: $Student \bowtie (Department \bowtie Course)$

Here, grouping of joins is changed for better efficiency.

5. Combine Selections

Original Expression: $\sigma_{age > 20}(\sigma_{dept='CS'}(Student))$

Transformed Form: $\sigma_{age > 20 \wedge dept='CS'}(Student)$

Here, multiple conditions are merged into one selection.

Query Optimization Strategies (methods / approaches):

The query optimization strategies are given as:

1. Cost-based query optimization
2. Heuristic Query Optimization
3. Nested Query Optimization

Cost-based Query Optimization:

Cost-based query optimization is a method used by the DBMS to choose the most efficient way to execute a query. It does this by generating multiple possible execution plans and estimating the cost of each plan in terms of CPU usage, disk I/O, and memory.

To estimate these costs, the optimizer uses statistical information from the database such as table size, number of records, indexes, and data distribution. After comparing all possible plans, it selects the one with the lowest estimated cost, which is expected to execute the query most efficiently.

Example:

For the query `SELECT * FROM Student WHERE age > 20;`, the DBMS may choose between scanning the whole table or using an index on the age column. It estimates the cost of both methods and finds that the index scan is cheaper because it accesses only required rows. So, it selects the index-based plan as it has the lowest cost and executes the query more efficiently.

Heuristic Query Optimization:

Heuristic query optimization is a rule-based approach used by the DBMS to improve query performance without calculating exact cost. Instead of trying all possible execution plans, it applies a set of general rules (heuristics) to rewrite the query into a more efficient form. These rules are based on common best practices, such as reducing the size of intermediate results early in the query execution.

Some common heuristic rules are:

-  Perform **selection and projection as early as possible** to reduce data size.
-  Apply the **most restrictive conditions first** to filter maximum rows early.
-  Avoid **Cartesian product (cross join)** because it creates very large intermediate results.
-  Reorder operations to reduce unnecessary processing.

Example:

Before Optimization	After Optimization
<pre>SELECT S.name FROM Student S, Department D WHERE S.dept_id = D.dept_id AND S.age > 20 AND D.dept_name = 'CS';</pre>	<pre>SELECT S.name FROM Student S, Department D WHERE S.age > 20 AND D.dept_name = 'CS' AND S.dept_id = D.dept_id;</pre>

Explanation:

Before optimization, the DBMS performs the join first and then applies selection conditions, which is inefficient because it creates a large intermediate result. After heuristic optimization, the DBMS first applies selection conditions like `age > 20` and `dept_name = 'CS'` to filter rows in both tables, and then performs the join on this reduced data, which reduces the size of intermediate results and makes query execution faster and more efficient.

Nested Query Optimization:

Nested query optimization is the process of improving the execution of subqueries (queries inside another query) so that they run more efficiently. A nested query is usually written inside the WHERE clause and its result is used by the main query.

The DBMS optimizes nested queries by trying to reduce repeated execution of the inner query. In many cases, it transforms the subquery into a join operation, which is faster. It may also apply techniques like query rewriting, pushing selections, and eliminating unnecessary computations.

Example:

Before Optimization	After Optimization
<pre>SELECT name FROM Student WHERE dept_id IN (SELECT dept_id FROM Department WHERE dept_name = 'CS');</pre>	<pre>SELECT Student.name FROM Student JOIN Department ON Student.dept_id = Department.dept_id WHERE Department.dept_name = 'CS'; -- Improved form using JOIN</pre>

Explanation:

In the optimized version, the subquery is converted into a JOIN operation, so the DBMS does not execute the inner query separately. Instead, both tables are processed together, which reduces repeated work and improves performance.

Unit 5

Transaction Management

Transaction:

A transaction in a database is a sequence of one or more operations (like insert, update, delete, or select) that are treated as a single logical unit of work. It ensures that either all operations are completed successfully or none are applied to the database.

ACID Properties:

ACID is a set of four properties that ensure database transactions are processed reliably and safely, even in case of errors, power failures, or concurrent access. The ACID properties include:

-  Atomicity
-  Consistency
-  Isolation
-  Durability

Atomicity:

Atomicity means a transaction is treated as a single unit. It is either completed fully or not executed at all. If any part of the transaction fails, the entire transaction is rolled back, and the database remains unchanged. This prevents partial updates.

Consistency:

Consistency ensures that a transaction takes the database from one valid state to another valid state. It maintains all rules, constraints, and integrity conditions of the database. This guarantees that the data remains correct and valid both before and after the transaction

Isolation:

Isolation ensures that multiple transactions can occur at the same time without interfering with each other. Each transaction executes independently, and intermediate changes of one transaction are not visible to others until it is committed. This prevents data conflicts in concurrent processing.

Durability:

Durability guarantees that once a transaction is successfully completed (committed), its changes are permanent. Even if the system crashes immediately after, the changes will not be lost because they are stored permanently in the database using recovery mechanisms.

Transaction Example:

Suppose you transfer ₹1000 from Account A to Account B. This process is a single transaction with two steps (operations):

-  1000 NR is deducted from Account A
-  1000 NR is added to Account B

Both steps must be completed successfully for the transaction to be valid. If the system fails after deducting money from Account A but before adding it to Account B, the DBMS will undo (roll back) the entire transaction. This ensures that no money is lost or incorrectly transferred, keeping the database consistent and correct.

State Diagram of Transaction:

[or, Life cycle of a Transaction]

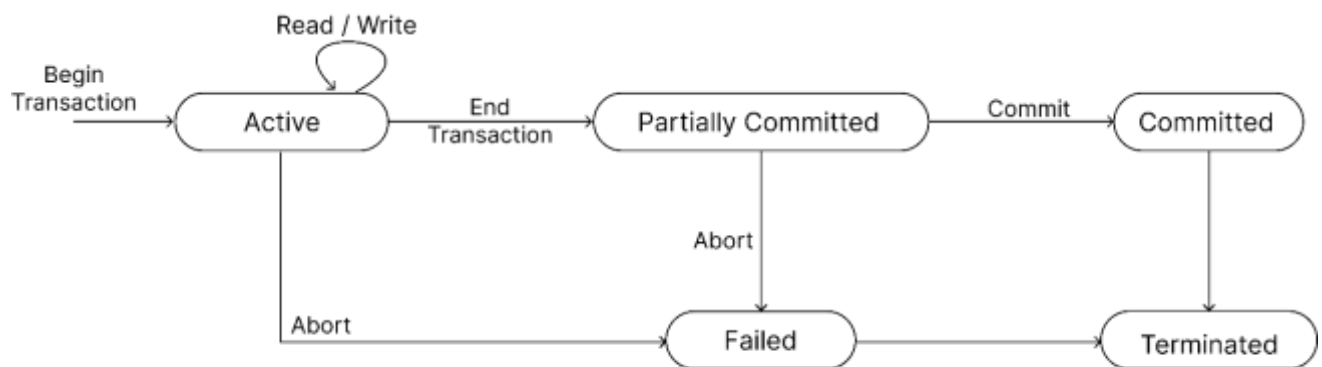


Fig: State Diagram of Transaction

1. Active State:

This is the first stage of a transaction when it starts executing. All operations like insert, delete, and update are performed here. The changes are kept temporarily in memory (buffer) and are not yet saved in the database.

2. Partially Committed State:

In this state, all operations of the transaction are completed, but the changes are still not permanently saved in the database. The data is stored temporarily. If everything goes fine, it moves to the committed state; otherwise, it goes to the failed state.

3. Committed State:

A transaction enters this state when all operations are successfully completed and the changes are permanently saved in the database using the COMMIT command. This means the transaction is successfully finished.

4. Failed State:

If any error occurs during execution (in active or partially committed state), the transaction stops and enters the failed state. The DBMS then checks the problem and prepares to restore the database to a correct state.

5. Aborted State:

In this state, the failed transaction is rolled back and all its changes are removed to keep the database consistent. The system may either:

- ✚ Stop the transaction completely, or
- ✚ Restart it again after fixing the issue

6. Terminated State:

This is the final state of a transaction. It means the transaction has finished its execution completely, either successfully (committed) or unsuccessfully (aborted), and no further action is needed.

Transaction Management:

The process of controlling and managing database transactions to ensure they are executed safely and correctly is called transaction management. It uses control mechanisms like concurrency control (to manage multiple transactions at once) and recovery techniques (to restore data after failures) to ensure the ACID properties are maintained.

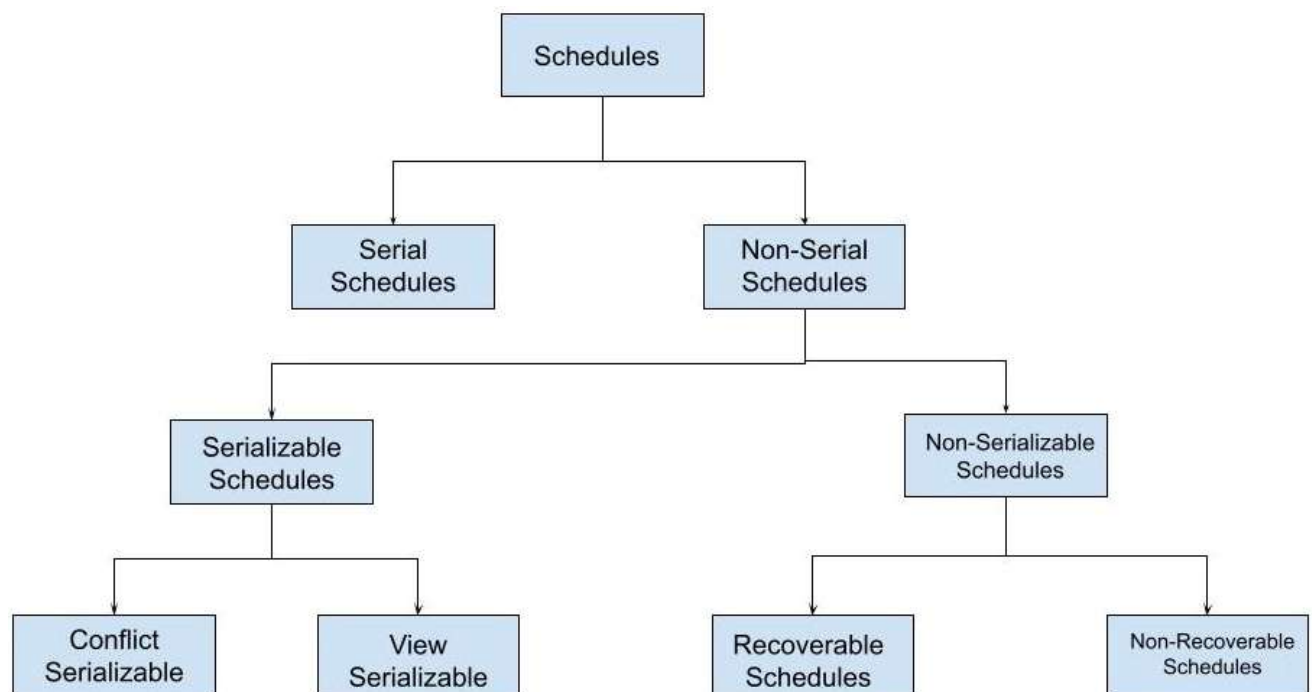
Concurrent Execution:

Concurrent execution means multiple transactions are executed at the same time on a database. In a multi-user system, different users can access data and perform operations on the same database simultaneously. This improves efficiency, but it can also create problems with data consistency if transactions are not properly managed. Therefore, DBMS uses control techniques to ensure that concurrent transactions do not interfere with each other.

Schedules:

A schedule is an ordered sequence of operations from one or more transactions. It shows the exact order in which read, write, and other operations are executed in the database. The main purpose of a schedule is to maintain the correct order of operations within each transaction while managing multiple transactions together.

The various types of schedules are:



Serial Schedule:

A serial schedule is a type of schedule in which transactions are executed one after another, without overlapping. In this type, a new transaction starts only after the previous one has completely finished. This means there is no concurrency, so it is very simple and always maintains data consistency.

T ₁	T ₂
R(A) W(A) R(B)	W(B) R(A) R(B)

Explanation:

First, T₁ executes completely (read and write operations on A and B). After T₁ finishes, T₂ starts execution. There is no overlap between T₁ and T₂, so this is a serial schedule.

Non-Serial Schedule:

It is a type of scheduling in which operations of multiple transactions are overlapped i.e. the transactions are executed in an overlapping manner. Here, a transaction does not need to wait for another transaction to complete fully. This increases concurrency and system efficiency, but it may also lead to consistency issues if not properly managed.

Non-serial schedules are further classified into:

[Schedule types based on Serializability]

1. Serializable
2. Non-serializable

Serializable Schedule:

A serializable schedule is a type of non-serial schedule whose result is equivalent to a serial schedule. This means that even though transactions are executed in an interleaved (concurrent) manner, the final outcome of the database remains the same as if the transactions were executed one after another. It ensures correctness and consistency in concurrent transaction execution, making it the most important type of schedule used in DBMS. A serializable schedule helps in improving resource utilization and CPU throughput.

T ₁	T ₂
R(A)	R(B)
W(A)	W(B)

Explanation:

For example, T₁ operates on A and T₂ on B, and their operations may overlap, but since they don't conflict, the final result is the same as executing T₁ and T₂ one after another.

Non-serializable Schedule:

A non-serializable schedule is a type of interleaved (non-serial) schedule in which the execution of transactions does not produce the same result as any serial execution of those transactions. In this case, the operations of different transactions conflict in such a way that the final outcome becomes incorrect or inconsistent. Because of this, non-serializable schedules are not considered safe in DBMS and may lead to problems like lost updates or data inconsistency.

To solve issues caused by non-serializable schedules, DBMS uses concurrency control techniques such as locking and Two-Phase Locking (2PL). These methods control how transactions access data and ensure they do not interfere with each other. This helps maintain serializability and prevents problems like data inconsistency and lost updates.

Non-serializable Schedules are further classified into:

[Schedule types based on Recoverability]

1. Recoverable Schedule
2. Non-recoverable Schedule

Recoverable Schedule:

A schedule in which a transaction is allowed to commit only after all the transactions it depends on have successfully committed is called a recoverable schedule. In other words, if one transaction reads data written by another transaction, it must wait for that transaction to commit first. This ensures that if a failure occurs, the database can be recovered without using uncommitted or incorrect data, maintaining consistency and reliability.

Non-recoverable Schedule:

A schedule where a transaction commits before the transaction it depends on has committed is known as non-recoverable schedule. This means it may commit after reading data from an uncommitted transaction. If the first transaction fails later, the database becomes inconsistent and cannot be safely recovered.

COMMIT and ROLLBACK:

COMMIT and ROLLBACK are transaction control statements used to make changes permanent or undo changes in a database.

Difference Between:

COMMIT	ROLLBACK
➤ COMMIT is a transaction control command used to permanently save all changes made during a transaction in the database.	➤ ROLLBACK is a transaction control command used to undo all changes made during a transaction.
➤ It makes the changes permanent once the transaction is successfully completed.	➤ It cancels the changes when a transaction fails or is not completed successfully.
➤ After COMMIT, the modified data becomes a part of the database and cannot be undone.	➤ After ROLLBACK, the database is restored to its previous consistent state before the transaction started.
➤ It is used when the transaction is executed successfully without errors.	➤ It is used when an error occurs or the user wants to cancel the transaction.
➤ It ensures that successful transactions are permanently stored.	➤ It ensures that incorrect or incomplete transactions do not affect the database.

Unit 6

Concurrency Control

Concurrency:

Concurrency refers to the execution of multiple transactions simultaneously. It allows several users to access and manipulate the database at the same time, improving system efficiency and resource utilization. When multiple transactions are executed simultaneously, there occurs data conflicts and inconsistencies.

The common problems with concurrency are:

- ❖ **Dirty Read:** Occurs when a transaction reads data that has been modified by another transaction but not yet committed. If the changes are later rolled back, the read value becomes invalid.
- ❖ **Non-Repeatable Read:** Occurs when a transaction reads the same data twice and gets different values because another transaction updated and committed the data in between the reads.
- ❖ **Phantom Read:** The problem that occurs when a transaction T1 reads a data item from the database and another transaction T2 deletes that data item from the main memory and the T1 doesn't find that data item in the main memory.

These types of problems can be solved using concurrency control.

Concurrency control:

Concurrency Control in a Database Management System (DBMS) is the technique of managing multiple transactions executing simultaneously without interfering with one another. It ensures that concurrent transactions are processed correctly and consistently while maintaining the integrity and accuracy of the database. Concurrency control also guarantees important transaction properties such as atomicity, isolation, and serializability.

The importance / Advantages of concurrency control are:

- ❖ Allows multiple users and transactions to access the database simultaneously without conflicts.
- ❖ Maintains data accuracy, consistency, and integrity.
- ❖ Prevents problems such as lost updates, dirty reads, and inconsistent data.
- ❖ Ensures transactions are executed in an isolated and reliable manner.
- ❖ Preserves important transaction properties like atomicity and serializability.
- ❖ Improves the overall performance and efficiency of the database system.
- ❖ Increases the reliability and correctness of transaction processing.

Types of Concurrency control techniques:

1. Lock-Based Protocols
2. Timestamp-Based Protocols.
 - a. Timestamp Ordering Protocol
 - b. Thomas Write Rule
3. Validation-Based Protocols (*also called Optimistic Concurrency Control*)
4. Multi-Version Concurrency Control (MVCC) / Multi-version Protocol

Lock-Based Protocols:

Lock-based protocols in DBMS are a concurrency control mechanism in which a transaction is not permitted to perform read or write operations on a data item unless it first acquires the required lock on that item. This ensures controlled access to shared data and helps prevent inconsistencies arising from concurrent execution of transactions.

A lock is a synchronization variable associated with a data item that defines the type of access allowed on it. It regulates how transactions interact with shared database resources in a multi-user environment. Locks are broadly classified into 2 types. They are:

1. Binary Lock:

A binary lock has two states: **locked** and **unlocked**.

- ❖ When a data item is locked, no other transaction can access it.
- ❖ When it is unlocked, it is available for access by a transaction.

2. Shared / Exclusive Lock:

This locking scheme provides finer control over concurrent access:

❖ Shared Lock (S-lock):

Also referred to as a read-only lock. Multiple transactions can simultaneously hold a shared lock on the same data item, as they are restricted to read operations only and cannot modify the data.

❖ Exclusive Lock (X-lock):

Used when a transaction intends to perform a write operation. A transaction holding an exclusive lock has both read and write privileges on the data item, while preventing any other transaction from acquiring either a shared or exclusive lock on the same item. The exclusive lock is requested using a lock-x instruction and is released once the write operation is completed.

Lock-Based protocols are broadly classified based on how and when locks are acquired and released by transactions to control concurrency access. The types are:

1. Simplistic Lock Protocol
2. Two-Phase Locking (2PL) Protocol
3. Strict Two-Phase Locking (Strict 2PL) Protocol
4. Rigorous Two-Phase Locking

Simplistic Lock Protocol:

In this protocol, a transaction must acquire a lock on a data item before performing any read or write operation. The lock is released immediately after the operation is completed. It is easy to implement but does not guarantee complete consistency in concurrent execution.

Two-Phase Locking (2PL) Protocol:

This is a widely used protocol that ensures serializability by dividing a transaction into two phases:

- ❖ Growing Phase: The transaction can acquire locks but cannot release any lock.
- ❖ Shrinking Phase: The transaction can release locks but cannot acquire new ones.

Once a transaction starts releasing locks, it cannot obtain any new locks. This structure ensures proper ordering of transactions and avoids conflicts.

Strict Two-Phase Locking (Strict 2PL) Protocol:

In this protocol, a transaction keeps all its exclusive (write) locks until it either commits or aborts. This prevents other transactions from reading or writing uncommitted data, thereby avoiding problems like dirty reads and ensuring strong consistency.

Rigorous Two-Phase Locking:

This is the strictest form of 2PL. Here, a transaction holds both shared (read) and exclusive (write) locks until it commits or aborts. It provides maximum isolation and ensures complete serializability of transactions.

Time-Stamp Based Protocols:

Timestamp-based protocol is a concurrency control technique in DBMS where each transaction is assigned a unique timestamp, and all operations are executed according to this timestamp order. The DBMS ensures that older transactions are given priority over newer ones, maintaining a correct and serializable execution sequence. If any operation violates the timestamp order rules, the transaction is aborted and restarted to preserve consistency.

It has following methods:

1. Timestamp ordering Protocol
2. Thomas Write Rule

Timestamp Ordering Protocol:

Timestamp Ordering Protocol is a concurrency control method in DBMS that ensures transactions are executed in a serializable order based on their timestamps. Each transaction is assigned a unique timestamp when it enters the system, and all read and write operations are performed according to this timestamp order.

Each data item maintains two values:

-  **Read Timestamp (RTS):** Latest timestamp of transaction that performed read operation successfully on data item
-  **Write Timestamp (WTS):** Latest timestamp of transaction that performed write operation successfully on data item.

Working principle:

- ❖ A read operation is allowed only if the transaction's timestamp $\geq$ WTS of the data item; otherwise, the transaction is aborted.
- ❖ A write operation is allowed only if the transaction's timestamp $\geq$ both RTS and WTS of the data item; otherwise, the transaction is aborted and restarted.

Conclusion:

The Timestamp Ordering Protocol guarantees serializability without using locks, but it may lead to higher transaction rollbacks in case of conflicts.

Thomas Write Rule:

Thomas Write Rule is an enhancement of the Timestamp Ordering Protocol. It improves efficiency by reducing unnecessary transaction aborts.

In this rule, if a transaction attempts to perform a write operation on a data item but its timestamp is older than the current Write Timestamp (WTS) of that item, the write operation is simply ignored

(discarded) instead of aborting the entire transaction. This is because the write is considered outdated and no longer relevant. If the timestamp condition is valid, the write operation is executed normally and the Write Timestamp is updated.

Conclusion:

Thomas Write Rule enhances concurrency by avoiding unnecessary rollbacks & allows more efficient transaction processing.

Validation Based Protocol:

[or, Optimistic Concurrency Control]

It is a concurrency control technique where transactions execute without strict synchronization during their execution. It assumes that conflicts between transactions are rare and checks for consistency only before committing.

In this approach, transactions do not directly update the database during execution. Instead, all changes are made in a local copy. The actual database is updated only after successful validation, which helps reduce locking and interference between transactions.

The phases of Validation-based protocol are:

1. Read Phase:

In this phase, a transaction reads data from the database and performs all required operations using a local copy of the data on local memory. Any updates or changes are stored temporarily in the local memory and do not affect the actual database.

2. Validation Phase:

In this phase, the system checks whether the transaction can be safely committed. It verifies that the transaction's execution does not conflict with other concurrently executing transactions and that committing it will not violate serializability. If any conflict is detected, the transaction is rejected.

3. Write Phase:

If validation is successful, all changes made in the local copy are written to the database. If validation fails, the transaction is aborted and rolled back.

Conclusion:

Validation-Based Protocol allows high concurrency by minimizing locking during execution. However, it may lead to transaction rollbacks if conflicts are detected during the validation phase.

Granularity:

Granularity refers to the size or level of a data item that can be locked during concurrency control. It defines how large or small a data unit (such as a database table, row, or field) is when a lock is applied.

The levels of granularity are:

- 1. Database Level:** Entire database is locked.
- 2. Table Level:** Entire table is locked.
- 3. Page (Block) Level:** A disk page containing multiple records is locked.
- 4. Row (Record) Level:** A single row of a table is locked.
- 5. Field (Attribute) Level:** A single attribute/column value is locked.

Multiple Granularity is a locking technique where data items are arranged in a hierarchy, & transactions are allowed to lock data at different levels such as database, table, page, or row to improve flexibility and concurrency control.

Multi-Version Concurrency Protocol (MVCC):

[Multi-version Protocol]

MVCC is a concurrency control technique in DBMS in which multiple versions of a data item are maintained simultaneously. Instead of updating an existing data item directly, each write operation creates a new version. This approach allows transactions to access a consistent snapshot of the database without interfering with one another.

Working principle:

- ✚ A **read operation** retrieves the appropriate version of a data item, usually the most recent committed version that is valid for that transaction.
- ✚ A **write operation** does not overwrite existing data; instead, it creates a **new version** of the data item.
- ✚ Since different versions exist, read and write operations can proceed concurrently without blocking each other.

Conclusion:

MVCC enhances concurrency and system performance by allowing simultaneous execution of transactions. It ensures consistency by enabling reads on older committed versions while writes generate new versions.

Deadlock:

A deadlock is a situation where two or more transactions are permanently blocked because each transaction is waiting for a resource (lock) held by another transaction. As a result, none of the transactions can proceed, causing the system to remain in a waiting state indefinitely. Deadlock is considered a serious problem in concurrency control because it prevents the completion of transactions and brings progress to a halt.

Example:

Consider two transactions, **T1** and **T2**. T1 holds a lock on some rows in the *Student* table and requests a lock on the *Grade* table, while T2 holds a lock on some rows in the *Grade* table and requests a lock on the *Student* table. As both transactions wait for each other to release their locks, a circular waiting condition is formed, resulting in a deadlock where neither transaction can proceed.

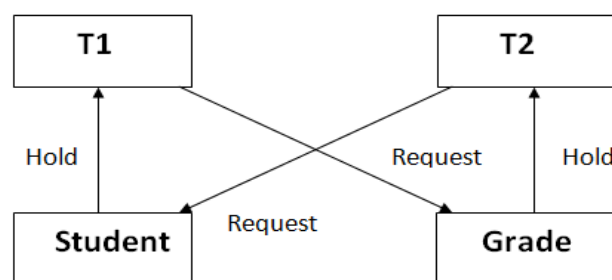


Fig: Deadlock in DBMS

Deadlock Prevention:

Deadlock prevention is a technique used to ensure that a system never enters a deadlock state by carefully controlling how resources (locks) are allocated to transactions. In this approach, the DBMS analyzes each transaction before execution and ensures that the allocation of locks will not lead to a deadlock situation. If a potential deadlock is detected, the transaction is not allowed to proceed.

Deadlock prevention is more suitable for large databases where avoiding deadlocks is critical for system stability.

The deadlock prevention schemes are:

1. Wait-Die Scheme:

Wait-Die is a non-preemptive deadlock prevention scheme based on timestamp ordering. In this method, if an older transaction requests a lock held by a younger transaction, the older transaction is allowed to wait.

However, if a younger transaction requests a lock held by an older transaction, the younger transaction is aborted (dies) and later restarted with the same timestamp. This ensures that only older transactions are allowed to wait, while younger ones are rolled back when a conflict occurs.

2. Wound-Wait Scheme:

Wound-Wait is a preemptive deadlock prevention scheme. In this method, if an older transaction requests a lock held by a younger transaction, the younger transaction is aborted (wounded) and the lock is granted to the older transaction.

On the other hand, if a younger transaction requests a lock held by an older transaction, the younger transaction is made to wait. This approach gives priority to older transactions and reduces unnecessary waiting for them, thereby preventing deadlock situations.

Deadlock Avoidance:

Deadlock avoidance is a technique in DBMS where the system prevents deadlocks by making careful decisions before allocating resources. Instead of reacting to deadlocks, it checks in advance whether granting a resource request will lead to an unsafe state. This method requires prior knowledge of the maximum resources each transaction may need, including future requests and releases. Based on this information, the DBMS allocates resources only if the system remains in a safe state.

The main limitation of this approach is accurately predicting the future resource needs of transactions before execution begins. A common deadlock avoidance method is the **Banker's Algorithm**, which grants resource requests only when the system remains safe.

Deadlock Detection:

If deadlock prevention and avoidance techniques are not properly applied, the system may enter a deadlock state. In such cases, the DBMS must detect the deadlock and take corrective action to recover the system.

One common method for deadlock detection is the **Wait-for Graph**. In this method, each transaction is represented as a vertex in a graph, and a directed edge from one transaction to another indicates that the first transaction is waiting for a resource held by the second. If the graph contains a cycle, it indicates the presence of a deadlock.

The wait-for-graph below represents a deadlock situation because process P1 is waiting for the resource held by p2, p2 is waiting for the resource held by p3, and so on. As we can see, there is a cycle.

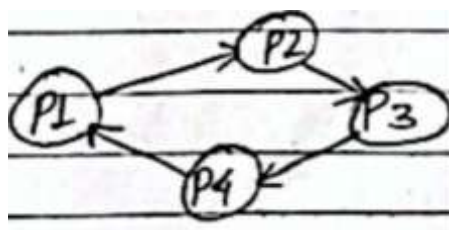


Fig: Wait for Graph

Recovery from Deadlock:

Once a deadlock is detected, the system must take action to recover from it & allow normal transaction processing to continue. Recovery involves breaking the circular waiting condition by removing one or more transactions involved in the deadlock. The most common method of recovery is transaction termination (abort).

The methods of deadlock recovery are:

1. Transaction Abortion (Termination):

In this method, the DBMS selects one or more transactions involved in the deadlock as victims and aborts them. Aborting a transaction releases all the locks it holds, allowing other transactions to proceed. The victim is chosen based on criteria such as transaction priority, execution cost, or amount of work completed.

2. Partial Rollback:

In partial rollback, a transaction is not fully aborted. Instead, it is rolled back to a previous consistent state (checkpoint), and then restarted from that point. This helps reduce loss of already completed work while still resolving the deadlock.

Starvation:

Starvation is a situation where a transaction is forced to wait for an indefinite period of time before it can acquire the required lock or access a resource. It occurs when a transaction is repeatedly denied access because other transactions are continuously given priority.

Some reasons of starvations are:

-  **Unfair scheduling policy:** When the system uses priority-based scheduling, lower-priority transactions may be repeatedly delayed in favor of higher-priority ones.
-  **Repeated victim selection:** During deadlock handling, the same transaction may be selected multiple times for rollback, preventing it from making progress.
-  **High resource contention:** When many transactions continuously hold required locks, some transactions may be forced to wait for a long time.
-  **Excessive load or denial-of-service condition:** A transaction may experience prolonged delay due to a large number of competing requests from other transactions.

Note:

Starvation and deadlock are different in nature. In starvation, there is no circular waiting; a single transaction keeps waiting for a long time while other transactions continue to execute normally. In deadlock, two or more transactions are stuck in a circular waiting condition, where each transaction waits for a resource held by another, and none of them can proceed unless the deadlock is resolved.

Unit 7

Recovery System

Database Recovery:

The process of restoring a database to a consistent state after a failure is called database recovery. The main objective of recovery techniques is to maintain data integrity and consistency, ensuring that the database is brought back to a correct state and that data loss is minimized or completely prevented.

Types of Failure in DBMS:

A Database Management System (DBMS) is vulnerable to different kinds of failures. To identify and handle problems effectively, failures are generally classified into the following categories:

1. Transaction Failure
2. System Crash
3. Disk Failure

Transaction Failure:

A transaction failure occurs when a transaction cannot complete its execution or reaches a point where it cannot proceed further. In this case, only the affected transaction is impacted, not the entire system. The reasons for transaction failure are:

Logical errors:

Occur when a transaction cannot complete due to incorrect logic, invalid input, or internal processing errors.

System-enforced aborts (Ex: resource issues):

The DBMS may terminate a transaction due to deadlock, insufficient resources, or inability to execute the operation.

System Crash:

A system crash happens when the database system stops working due to hardware or software failure, such as a power outage or operating system error.

 During a system crash, all active transactions are lost.

 It is generally assumed that non-volatile storage (like disks) remains intact and is not corrupted.

Disk Failure:

Disk failure occurs when the storage device used by the DBMS becomes damaged or inaccessible. Disk failure can result in partial or complete loss of stored data. Some common causes of disk failure include:

-  Formation of bad sectors
-  Disk head crash
-  Physical damage or corruption

Recovery Schemes:

Recovery schemes are techniques used by a DBMS to restore the database to a correct and consistent state after a failure. These techniques ensure database reliability and maintain the ACID properties, especially atomicity and durability.

When a system failure occurs, some transactions may have been completed while others may have been remained incomplete. Therefore, recovery system checks the status of all transactions and decides whether the transactions should be redone or undone so that the database does not remain in an inconsistent state.

The objectives of Recovery Schemes are:

- ✚ Restore the database after a failure.
- ✚ Maintain data consistency.
- ✚ Ensure atomicity of transactions.
- ✚ Preserve committed data.
- ✚ Undo incomplete transactions.

The main Recovery Techniques are:

1. Log Based Recovery
2. Shadow Paging

Log Based Recovery:

In this technique, every operation performed by a transaction is first recorded in a special file called a log file before the actual database is updated. The log file is stored on stable storage so that it remains safe even if the system crashes. Log-based recovery ensures the atomicity and durability properties of transactions.

Recovery in this technique is performed using the information stored in the log file. During transaction execution, the DBMS records all transaction activities such as transaction start, data modification, commit, or abort in the log file. If a system failure occurs, the recovery system examines the log records to determine which transactions were completed and which remained incomplete. Committed transactions are redone to preserve their changes, while incomplete transactions are undone to restore the database to its previous consistent state.

Log-based recovery follows an important principle called **Write Ahead Logging (WAL)**. According to WAL, every log record must be written to stable storage before the corresponding changes are applied to the database. In other words, the DBMS first records all modification details in the log file and only then performs the actual update on the database. This ensures database is correctly restored to a consistent state using the logs in case of failure. WAL therefore guarantees that no transaction updates are lost.

The operations in log-based recovery are:

1. REDO Operation:

Redo means reapplying the changes of a committed transaction to ensure its updates are permanently reflected in the database.

2. UNDO Operation:

Undo means reversing the changes of an incomplete or failed transaction to restore the database to its previous consistent state.

Log File:

A log file is a special file in DBMS that records all operations performed by transactions, such as start, updates, commit, and abort. It is stored in stable storage to ensure it is not lost during system failure. During recovery, the DBMS uses the log file to redo committed transactions and undo incomplete transactions, thereby maintaining database consistency.

Checkpoint:

A checkpoint represents a consistent state of the database, meaning all the transactions completed before it are safely stored on disk. Checkpoint improves the recovery efficiency after a system failure. At the time of a checkpoint:

- ✚ All modified data in memory (buffer) is written to the database on the disk
- ✚ A checkpoint record is added to the log file
- ✚ Transactions completed before the checkpoint are considered safely stored.

Recovery using Checkpoint:

[or, Checkpointing]

Checkpoint recovery is a technique used to recover a database efficiently after a system failure by using a checkpoint record in the log file. A checkpoint represents a consistent state of the database, meaning all transactions completed before it are safely stored on disk. During recovery, the DBMS does not scan the entire log file. Instead, it starts from the latest checkpoint, which reduces recovery time and effort.

The working of checkpoint recovery mechanism is given as:

After a crash, the DBMS performs the following steps:

1. Locate the last checkpoint in the log file.
2. Identify transactions after the checkpoint.
3. Create two lists:
 - a. Redo list → committed transactions
 - b. Undo list → incomplete transactions
4. Apply:
 - a. Redo for transactions in redo list
 - b. Undo for transactions in undo list

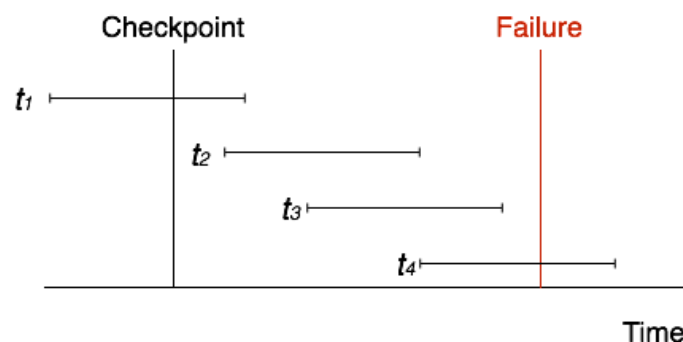


Fig: Recovery using checkpoint

The advantages of checkpoint recovery are:

- ✚ **Faster recovery:** DBMS does not check full log, only starts from last checkpoint.
- ✚ **Less processing:** Only transactions after checkpoint are analyzed.
- ✚ **Saves time:** Reduces redo and undo work during recovery.
- ✚ **Improves performance:** Recovery becomes simpler and more efficient.
- ✚ **Consistent restart point:** Database always restarts from a safe and consistent state.

Explanation for Notation Used on below example:

$\langle TX, A, 100, 200 \rangle$ Means transaction Tx changed data item A from 100 to 200. 100 is old value and 200 is new value for data item A.

Example:

[For Checkpoint recovery]

Log File
<T1, Start> <T1, A, 100, 200> <T1, Commit> <Checkpoint> <T2, Start> <T2, B, 300, 400> <T2, Commit> <T3, Start> <T3, C, 500, 600> ----- Failure Occur here -----

Suppose crash happens after T3 starts. Then recovery steps are:

1. Start from **Checkpoint**
2. Ignore all logs before checkpoint (T1 already safe, so its ignored)
3. T2 → Committed → REDO list
4. T3 → No Committed → UNDO list

Recovery is applied as:

1. REDO T2 → Ensure B = 400 is written to disk
2. UNDO T3 → Restore C from 600 back to 500

Final Result after recovery:

- ✚ T1 is already safely stored (ignored)
- ✚ T2 is successfully reapplied (redo)
- ✚ T3 is rolled back (undo)

Shadow Paging:

Shadow paging is a database recovery technique in which the database is divided into fixed-size blocks called pages, and a page table is used to map these pages to their physical locations in secondary storage.

In this method, 2 page tables are maintained:

1. Current Page Table

1. Used during transaction execution.
2. It is modified whenever updates are made.

2. Shadow Page Table

- ✚ Remains unchanged throughout the transaction.
- ✚ It represents the last consistent state of the database.
- ✚ It is stored in **non-volatile (stable) storage** for recovery purposes.

At the beginning of a transaction, both the current and shadow page tables are identical. During execution, any updates are not made directly to the original pages. Instead, new pages are created, and the current page table is updated to point to these new pages, while the shadow page table remains unchanged.

If the transaction completes successfully, the current page table replaces the shadow page table. However, if a failure occurs, the system discards the current page table and uses the shadow page table to restore the database to its previous consistent state.

The advantages of shadow paging are:

- + No need to maintain log files.
- + No need for undo or redo operations.
- + Recovery is fast and simple (just switch page tables).

The disadvantages of shadow paging are:

- + **Data fragmentation:**
Frequent creation of new pages causes data to be scattered across storage, reducing efficiency.
- + **High storage usage:**
Additional space is required since updated pages are stored separately instead of overwriting existing ones.
- + **Limited concurrency support:**
It is not well-suited for handling multiple transactions simultaneously.

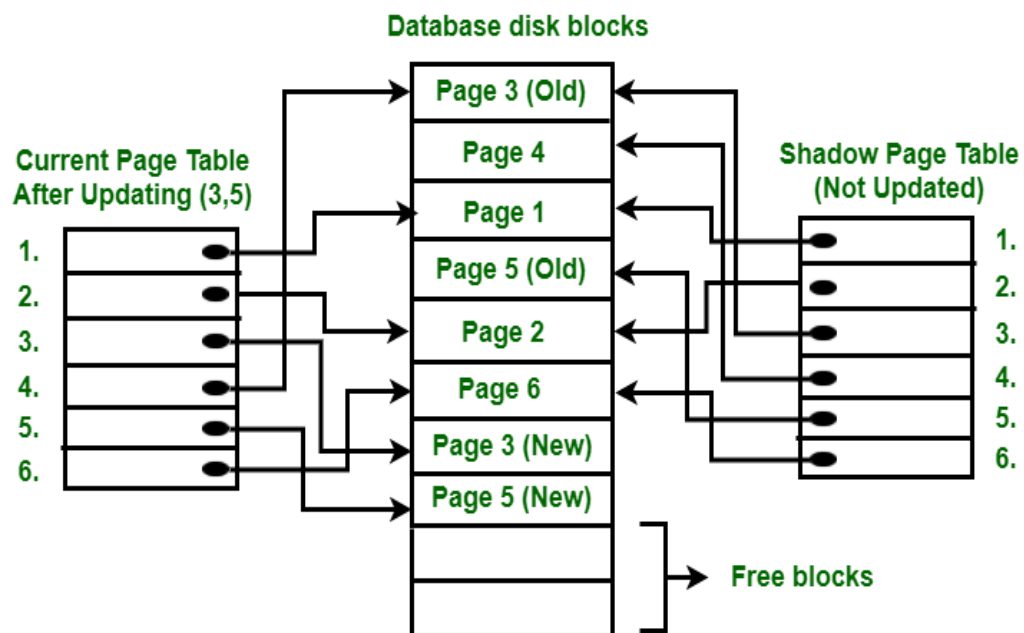


Fig: Shadow Paging

Recovery in Concurrency:

Recovery in concurrency refers to restoring the database to a correct and consistent state when multiple transactions are executing simultaneously and a system failure occurs. It is necessary because, in concurrent execution, transactions overlap and interleave, so a crash may leave some transactions partially completed and others incomplete, leading to an inconsistent database state.

Therefore, recovery ensures that all committed transactions are preserved and all incomplete transactions are properly undone to maintain consistency.

The ways of Recovery in concurrency are explained below:

1. Interaction with concurrency control:

In concurrent systems, recovery is closely linked with concurrency control methods like locking, timestamp ordering, or serializability rules. These methods ensure proper execution order of transactions. If a transaction fails, only that transaction is affected and rolled back, while other concurrent transactions that are not affected continue normally. This helps maintain consistency without disturbing the whole system.

2. Transaction Rollback:

Transaction rollback is used when a transaction fails before completion. In this method, all the changes made by the failed transaction are undone using log records. The DBMS restores the old values of data items, bringing the database back to its previous consistent state. Rollback ensures that partial updates of a failed transaction do not remain in the database.

3. Checkpointing:

Checkpointing is a technique where the DBMS saves a consistent state of the database. During recovery, the system does not check the entire log file; instead, it starts from the last checkpoint and processes only the transactions that occurred after it, making recovery faster and more efficient.

4. Restart Recovery:

Restart recovery is performed when the system restarts after a crash. The DBMS reads the log file and identifies the status of all transactions. It then creates two groups: a redo list for transactions that were completed (committed) and an undo list for transactions that were not completed. After that, all committed transactions are redone to ensure their changes are saved, and all incomplete transactions are undone to remove their partial effects. This restores the database to a correct and consistent state.

Missing QNs Topics

1. Explain data integrity and its importance in DBMS.

Data integrity refers to the accuracy, consistency, validity, and reliability of data stored in a database. It ensures that the data remains correct and consistent throughout its lifecycle and is modified only through authorized operations. A DBMS maintains data integrity using constraints, keys, rules, and validation techniques.

The importance of data integrity:

- Ensures that the data stored in the database is accurate and reliable.
- Prevents duplicate, invalid, and inconsistent data entries.
- Maintains consistency of data during insert, update, and delete operations.
- Improves the reliability and quality of the database system.
- Maintains proper relationships between tables through keys and constraints.

2. What is DBMS? List out functions of a DBMS.

A DBMS is software used to create, store, organize, manage, and retrieve data efficiently in a database. It acts as an interface between users and the database. The functions of a DBMS are:

- Stores and manages data efficiently.
- Retrieves and updates data when required.
- Provides data security and access control.
- Maintains data integrity and consistency.
- Controls data redundancy.
- Supports backup and recovery of data.
- Manages concurrent access by multiple users.
- Provides data sharing among users and applications.

3. Difference Between:

DBMS	RDBMS
➤ DBMS stands for Database Management System.	➤ RDBMS stands for Relational Database Management System.
➤ Data is stored in file-based or non-tabular format.	➤ Data is stored in tabular form (rows and columns).
➤ Does not strictly maintain relationships between data.	➤ Maintains relationships using primary and foreign keys.
➤ Limited or no support for normalization.	➤ Fully supports normalization to reduce redundancy.
➤ Higher data redundancy.	➤ Lower data redundancy due to structured design.
➤ Weaker support for integrity constraints.	➤ Strong enforcement of integrity constraints.
➤ Limited support for multiple users.	➤ Efficiently supports multiple users simultaneously.
➤ Examples: simple or legacy database systems.	➤ Examples: MySQL, Oracle, PostgreSQL.

4. Explain join dependency with example.

Join dependency means a table can be split into smaller tables, and when we join those tables back together, we get the original table without losing any data. It is mainly used in advanced normalization (In 5NF).

Example:

Original Table		
Employee	Project	Skill
Ram	P1	Java
Ram	P1	SQL
Ram	P2	Java
Sita	P1	SQL

We can decompose above table into 3 tables: employee_project , employee_skill and project_skill as:

exmployee_project		employee_skill		project_skill	
Employee	Project	employee	Skill	Project	Skill
Ram	P1	Ram	Java	P1	Java
Ram	P2	Ram	SQL	P1	SQL
Sita	P1	Sita	SQL	P2	Java

This shows a join dependency exists between these relations because the original table can be reconstructed by joining all three decomposed tables together.

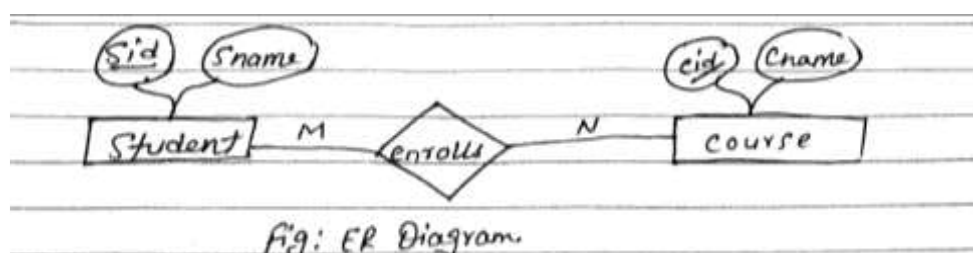
5. How can you covert an ER diagram into a table. Explain with example.

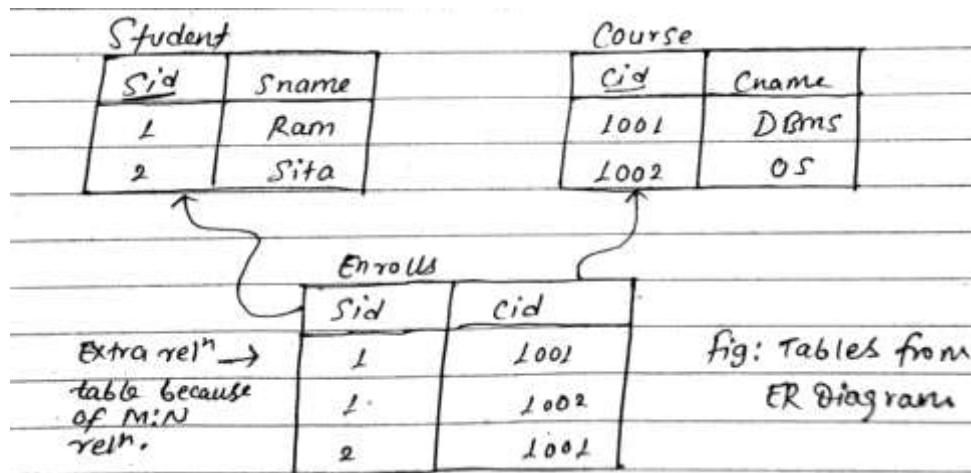
An ER (Entity–Relationship) diagram is converted into tables by mapping entities, attributes, and relationships into relational schemas in DBMS. Each entity becomes a table, and relationships are represented using primary keys and foreign keys.

The steps to convert ER Diagram into Tables are:

- 1. Entity → Table:**
Entity becomes table. Its attributes become columns, and the primary key remains the same.
- 2. 1:1 Relationship → Foreign Key:**
Add primary key of one table into another as a foreign key.
- 3. 1:M Relationship → Foreign Key:**
Add primary key of “one side” into “many side” table.
- 4. M:N Relationship → New Table:**
Create a separate table with both primary keys as foreign keys.
- 5. Multivalued attribute → Separate Table:**
Create a new table for multivalued attribute.

Example:





6. Explain data warehouse.

A data warehouse is a centralized storage system that stores data collected from multiple sources for analysis and decision-making. It contains historical and integrated data that is cleaned, organized, and stored separately from daily transaction systems. Unlike normal databases used for operations, a data warehouse is mainly used for reporting, analysis, and finding trends to support business decisions. For example, a company can combine sales, finance, and HR data in data warehouse to analyse overall performance and generate reports.

7. Difference between:

Physical Data Independence	Logical Data Independence
➤ It is the ability to change the physical storage of data without changing the logical structure.	➤ It is the ability to change the logical structure without changing user views or application programs.
➤ It focuses on how data is stored in memory (files, indexes, storage methods).	➤ It focuses on what data is stored (tables, attributes, relationships).
➤ Changes in storage structure do not affect the overall database design.	➤ Changes in table structure do not affect how users access data.
➤ It is easier to achieve in DBMS.	➤ It is harder to achieve in DBMS.
➤ Example: Changing file organization from sequential to indexed.	➤ Example: Adding a new column in a table without affecting existing programs.

8. How do you increase the performance of database tuning?

Database tuning is the process of improving database performance by optimizing queries, storage, and system resources so that data can be accessed faster and more efficiently. The different ways to tune database performance are:

- 🔧 **Proper Indexing:** Create indexes on frequently used columns to speed up data retrieval.
- 🔧 **Query Optimization:** Write efficient SQL queries and avoid unnecessary joins and subqueries.
- 🔧 **Use of Caching:** Store frequently accessed data in memory to reduce database access time.
- 🔧 **Normalization (Proper Level):** Reduce data redundancy, but avoid over-normalization that slows joins.

- ✚ **Efficient Table Design:** Use appropriate data types and structure tables properly.
- ✚ **Avoid Redundant Data:** Reduce duplication to improve storage and processing speed.
- ✚ **Database Partitioning:** Split large tables into smaller parts for faster access.
- ✚ **Proper Use of Keys:** Use primary and foreign keys correctly for faster joins.

9. Why do we use Outer Join?

We use outer join because:

- ✚ To include matching as well as non-matching records.
- ✚ To avoid losing data when there is no relationship between tables.
- ✚ To identify missing or unmatched records in tables.
- ✚ To get complete result sets from related tables.

10. What are the characteristics of a memory.

Memory in a computer system is used to store data and instructions. It has several important characteristics that define its performance and behaviour.

- ✚ **Speed:** Memory provides fast access to data and instructions.
- ✚ **Capacity:** Memory stores a large amount of data.
- ✚ **Volatility:** Memory may be volatile (data lost when power is off, Ex: RAM) or non-volatile (data remains stored, Ex: ROM).
- ✚ **Cost:** Faster memory is more expensive, while slower memory is cheaper.
- ✚ **Access Time:** It is the time required to read or write data in memory.
- ✚ **Type:** Memory is classified as primary memory (RAM, ROM) and secondary memory (HDD, SSD).

11. How can you recover data from loss?

Data can be recovered from loss using database recovery techniques that restore the system to a consistent state after failure. We can recover data in different ways:

- ✚ **Backup and Restore:**
Regular backups are taken, and data is restored from the latest backup when loss occurs.
- ✚ **Transaction Log (Log-based Recovery):**
DBMS keeps a log of all operations. It helps to redo or undo transactions during recovery.
- ✚ **Checkpointing:**
The system saves a consistent state at regular intervals. Recovery starts from the last checkpoint.
- ✚ **Rollback:**
Undo incomplete or faulty transactions to maintain consistency.
- ✚ **Redo Operation:**
Re-applies completed transactions that were not saved properly.

12. Is DBMS useful for research?

Yes, DBMS is very useful for research because it helps in storing, managing, and analysing large amounts of data efficiently. The Use of DBMS in research are:

- ✚ **Data Storage:** Stores large and complex research data in an organized way.
- ✚ **Easy Retrieval:** Helps quickly access required data for analysis.

- ✚ **Data Integrity:** Ensures accuracy and reliability of research data.
- ✚ **Data Security:** Protects sensitive research information from unauthorized access.
- ✚ **Efficient Analysis:** Supports queries that help in analyzing patterns and results.
- ✚ **Data Sharing:** Allows multiple researchers to access and work on the same data.

13. What is schema of a database?

A database schema is the overall structure or design of a database. It defines how data is organized in tables, including the tables, attributes (columns), relationships, and constraints. A database schema includes:

- ✚ Tables in the database
- ✚ Columns (attributes) in each table
- ✚ Data types of columns
- ✚ Relationships between tables
- ✚ Constraints like primary key and foreign key

14. Define foreign key?

A foreign key is a field (or set of fields) in one table that refers to the primary key of another table. It is used to create and maintain a relationship between two tables in a database.

15. What are the steps in query decomposition.

Query decomposition is the process of breaking a high-level query into smaller, more efficient operations that can be executed by the DBMS. The steps in query decomposition are:

1. **Parsing and translation:** The SQL query is checked for syntax errors and converted into an internal form such as relational algebra.
2. **Query normalization:** The query is rewritten into a simpler and standard form.
3. **Query simplification:** Unnecessary conditions and redundant operations are removed to improve efficiency.
4. **Query optimization:** The DBMS selects the most efficient execution plan based on cost (CPU, I/O, memory).
5. **Query execution:** The optimized query is executed to retrieve the final result from the database.

16. What are the advantages of using DBMS over the file processing system?

The advantages of DBMS over file processing system are:

- ✚ **Reduced data redundancy:** Avoids unnecessary duplication of data.
- ✚ **Improved data consistency:** Maintains the same and updated data across the system.
- ✚ **Better data sharing:** Allows multiple users to access data simultaneously.
- ✚ **Enhanced security:** Provides controlled access to data through authentication and authorization.
- ✚ **Data integrity:** Ensures accuracy and validity of stored data.
- ✚ **Faster data retrieval:** Uses indexing and queries for efficient access.
- ✚ **Backup and recovery:** Helps restore data in case of system failure.
- ✚ **Concurrency control:** Manages simultaneous access without data conflict.
- ✚ **Data independence:** Changes in structure do not affect application programs.

17. What are the functions of Buffer Manager?

A Buffer Manager is a DBMS component that manages data transfer between disk (secondary storage) and main memory (RAM). Its main goal is to reduce disk access and improve speed.

The functions of Buffer Manager are:

- 🚦 **Data movement:** Transfers data pages between disk and main memory.
- 🚦 **Buffer allocation:** Assigns space in memory to store data pages.
- 🚦 **Page replacement:** Replaces old pages with new ones when memory is full.
- 🚦 **Fast data access:** Keeps frequently used data in memory for quick retrieval.
- 🚦 **Write-back control:** Saves modified data from memory back to disk.
- 🚦 **Reduces disk I/O:** Minimizes the number of disk accesses to improve performance.

18. Difference Between:

B Tree	B+ Tree
➤ Data (keys and records) can be stored in both internal nodes and leaf nodes.	➤ Data (records) are stored only in leaf nodes.
➤ Internal nodes store both keys & data.	➤ Internal nodes store only keys (no data).
➤ Search can end at any level.	➤ Search always ends at leaf nodes.
➤ Leaf nodes are not linked together.	➤ Leaf nodes are linked for faster sequential access.
➤ Traversal is slower for range queries.	➤ Faster for range queries due to linked leaves.
➤ More space is used in internal nodes.	➤ More efficient use of internal node space.
➤ Example: Used where both search and direct access are needed.	➤ Example: Commonly used in databases for indexing.

19. Define primary and candidate key with examples.

A primary key is a column (or set of columns) that uniquely identifies each record in a table. It cannot have NULL values and must be unique.

A candidate key is any column (or set of columns) that can uniquely identify a record in a table. A table can have multiple candidate keys, but only one is chosen as the primary key.

20. How does bucket overflow occur and how do you handle it?

Bucket overflow occurs in hashing when more records are assigned to a bucket than it can hold. This happens because multiple keys generate the same hash value or when a bucket has limited space.

Bucket overflow occurs due to:

- 🚦 When many records map to the same bucket (collision).
- 🚦 When the bucket capacity is full.
- 🚦 When data is not evenly distributed across hash values.

Bucket overflow can be handled as:

- 🚦 **Overflow chaining (overflow bucket):**
Extra records are stored in a separate overflow area linked to the original bucket.
- 🚦 **Rehashing:** A new hash function is applied to redistribute records into different buckets.

- 🚦 **Bucket expansion:** The number of buckets is increased to reduce overflow chances.
- 🚦 **Linear probing:** If a bucket is full, the system checks the next available bucket and stores the record there.

21. Difference Between:

Sequential File Organization	Random (Direct) File Organization
➤ Records are stored in a sequential order (one after another).	➤ Records are stored in no fixed order.
➤ Data is accessed in a linear way (from beginning to end).	➤ Data can be accessed directly using a key or address.
➤ Slower for searching specific records.	➤ Faster for searching specific records.
➤ Suitable for batch processing.	➤ Suitable for real-time processing.
➤ Insertion and deletion are slow because reordering is required.	➤ Insertion and deletion are faster.
➤ Requires scanning multiple records to find data.	➤ Direct access reduces searching time.
➤ Example: Payroll systems, report generation.	➤ Example: Banking systems, reservation systems.

22. Why is lossless-join important?

Lossless-join is important because of following reasons.

- 🚦 Ensures **no** data loss during decomposition of tables.
- 🚦 Prevents creation of spurious (fake) tuples after joining tables.
- 🚦 Maintains data accuracy and correctness.
- 🚦 Helps in proper database normalization.
- 🚦 Ensures the original relation can always be reconstructed correctly.

23. What is normalization?

Normalization is the process of organizing data in a database to reduce redundancy and improve data consistency. It involves dividing large tables into smaller, related tables and defining relationships between them.

24. Explain 5NF with example.

Fifth Normal Form (5NF) is the highest level of normalization in DBMS where a table is decomposed into smaller tables so that it can be reconstructed correctly by joining them, without any loss of data or generation of incorrect tuples. It ensures proper handling of join dependency and eliminates redundancy caused by it.

Example:

Original Table		
Employee	Project	Skill
Ram	P1	Java
Ram	P1	SQL
Ram	P2	Java
Sita	P1	SQL

We can decompose above table into 3 tables: employee_project , employee_skill and project_skill as:

employee_project

Employee	Project
Ram	P1
Ram	P2
Sita	P1

employee_skill

employee	Skill
Ram	Java
Ram	SQL
Sita	SQL

project_skill

Project	Skill
P1	Java
P1	SQL
P2	Java

This shows that the original relation can be reconstructed by joining all three tables together. Hence, a join dependency exists. So, last tables are in Fifth Normal Form (5NF).
